

AART Slot Car Braking Module

Design Document

Target MCU: STM32G051K6U6 (QFN-32) | Firmware: Rev 5 | libopencm3

Revision 1.0 | AART — Adrian & Richard's Technologies

1. Overview

This document describes the hardware design and firmware architecture for the AART slot car braking module. The module connects in-line between a conventional hand controller and the track, providing variable resistance braking, positive anti-brake for brushed motors, and a keep-alive voltage floor for brushless motors. All three functions are controlled by a three-position toggle switch and a single linear potentiometer, with fine tuning via UART commands.

1.1 Purpose

The AART brake module provides three capabilities, controlled by a three-position toggle switch, a linear potentiometer, and UART commands:

- Braking — for brushed and brushless motors. Q2 shunts a variable resistance across the motor terminals. The pot governs brake force from a settable minimum (selectable 2/3/4/6/8Ω via SET BRAKE_OHMS, full CCW) to dead short (full CW). Fine electronic control replaces the fixed resistor of a conventional brake.
- Anti-brake — for brushed motors (Mode B). After the brake releases, Q1 injects a small forward voltage from the WHITE rail to counter the back-EMF that causes the car to snap on corner entry. The pot simultaneously governs both brake resistance and anti-brake level through a lookup table: lighter braking gives more anti-brake, harder braking less.
- Keep-alive floor — for brushless motors (Mode A with floor set). The eCom is powered from the braid. Conventional braking collapses the braid voltage to near zero, resetting the eCom. Q1 clamps the braid above a settable floor, preventing brownout. The floor is set by rotating the pot in Mode C then toggling to Mode A.

Parameters requiring fine or infrequent adjustment — ramp-in time, trail brake release, brake entry/exit thresholds, and Latvian brake — are set via UART and saved to flash.

The module is fully compatible with any conventional hand controller and requires no modification to the controller or the car.

1.2 System position

The module connects via 4mm banana plugs to the three standard track sockets and exports matching sockets for the hand controller to plug into. It is fully transparent during normal running and takes over braking when the controller shorts the track.

Banana colour	Signal	Description
WHITE	+12–16V always-live rail	From track PSU. Can deliver 40–90A. Powers the module and Q1 injection.
BLACK	Track positive (+ve to	Controlled by hand controller. Monitored by module to

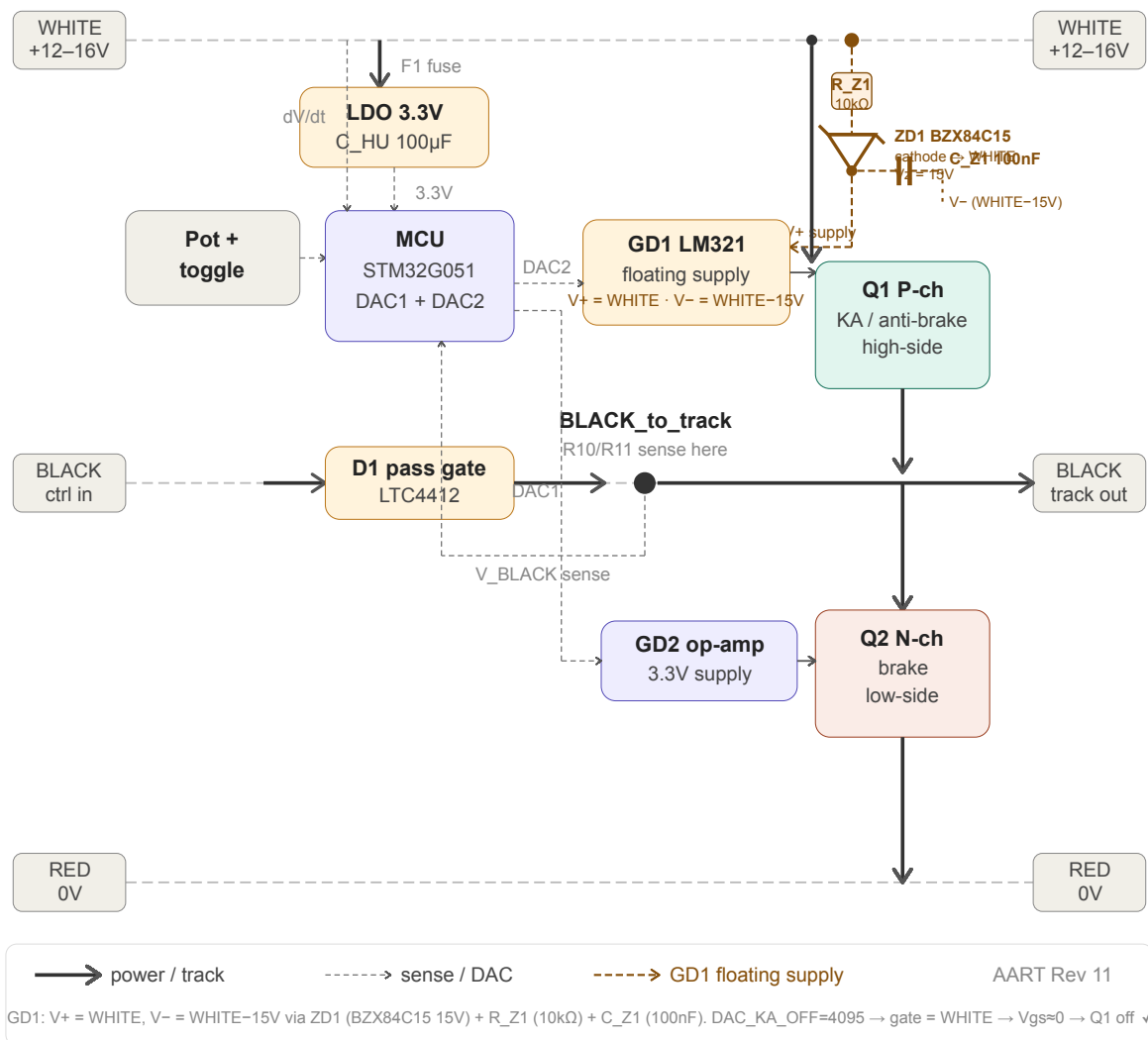
Banana colour	Signal	Description
	car)	detect brake condition.
RED	Common negative	Left braid return. Module and track ground reference.

The module presents identical WHITE/BLACK/RED sockets on its output side for the hand controller to plug into. During normal running, BLACK passes through unmodified. When the hand controller shorts BLACK→RED (brake condition), the module detects this by voltage sense on the BLACK terminal and applies its own controlled braking.

The WHITE rail is always live at 12–16VDC regardless of throttle position. The hand controller varies how much of this rail reaches the BLACK output. The module draws its MCU supply from WHITE via the LDO. WHITE is also the source for Q1 (anti-brake injection and keep-alive floor).

1.3 Circuit topology

The diagram below shows the principal signal paths through the module. All three rails (WHITE, BLACK, RED) run left to right. The hand controller connects on the left; the track and motor are on the right.



[Figure 1: Circuit topology diagram — see AART_Brake_Topology.svg delivered alongside this document]

Key points from the diagram:

- D1 (LTC4412 pass gate) sits in the BLACK controller input path — not the WHITE rail. During normal running it is fully transparent (near-zero voltage drop). During brake takeover it opens, isolating the controller's short from the track.
- Q1 (P-ch, high-side) connects WHITE to the BLACK_to_track node. In Mode A it applies the saved KA floor whenever V_{BLACK} drops below it. In Mode B it injects anti-brake voltage after brake exit. In Mode C it previews the KA floor live while the driver tunes it. Its gate is driven by a linear mode gate driver (op-amp buffer) from DAC2.

- Q2 (N-ch, low-side) connects BLACK_to_track to RED/GND. It provides the variable brake resistance from a selectable minimum (2/3/4/6/8Ω via SET BRAKE_OHMS) to a near dead short. Its gate is driven by a linear mode gate driver from DAC1.
- Both FETs operate in their linear (triode) region — no PWM switching. The DAC sets a DC gate voltage; the FET presents a continuously variable drain-source resistance. No 20kHz switching, no radiated EMI, no interference with brushless eCom electronics.
- The MCU senses WHITE dV/dt (for Latvian brake detection), V_BLACK_to_track (for brake entry/exit hysteresis), and the pot and mode switches. It generates DAC1 and DAC2 voltages accordingly.
- The R10/R11 voltage divider sensing BLACK_to_track is on the motor side of D1 — this is essential so the MCU can detect the voltage collapse that occurs when D1 opens and the module takes over braking.

2. Operating Modes

The module has three operating modes selected by a 3-position toggle switch. Within each mode the brake is triggered by voltage sense on the BLACK terminal rather than a mechanical wiper strip contact, making the module compatible with any hand controller.

2.1 Brake detection — voltage sense with hysteresis

The BLACK terminal voltage is continuously monitored via an ADC input. A hysteresis band prevents chatter at low throttle settings near zero:

- $V_BLACK < 500\text{mV} \rightarrow$ enter brake state (controller has shorted BLACK→RED)
- $V_BLACK > 1500\text{mV} \rightarrow$ exit brake state (controller delivering power)
- $500\text{mV} \leq V_BLACK \leq 1500\text{mV} \rightarrow$ hold current state

This approach works with any hand controller regardless of whether it has an accessible wiper strip contact, and correctly handles partial braking (controllers that do not short fully to zero).

2.2 Mode A — Brake only (brushed and brushless)

V_BLACK	Pot position	Q1 (KA floor)	Q2 (brake)	Behaviour
> 1500mV	Any	saved_ka_data if floor set, else OFF	OFF	Pass-through — module transparent. Q1 holds KA floor if configured.
< 500mV	Full CCW	OFF (during braking)	Configurable soft min (SET BRAKE_OHMS)	Softest brake at the selected resistance (2/3/4/6/8Ω).

V_BLACK	Pot position	Q1 (KA floor)	Q2 (brake)	Behaviour
< 500mV	CCW → CW	OFF (during braking)	Rising toward DAC_BRAKE_HARD	Brake firms progressively as pot turns CW.
< 500mV	Full CW	OFF (during braking)	DAC_BRAKE_HARD	Dead short — maximum braking.
< 500mV → released	Any	saved_ka_dac applied when V_BLACK < floor	OFF	Q1 maintains keep-alive floor (brushless). Default DAC_KA_OFF = pure braking.

2.3 Mode B — Brake and anti-brake (brushed motors)

V_BLACK	Pot position	Q1 (anti-brake)	Q2 (brake)	Behaviour
> 1500mV	Any	OFF	OFF	Pass-through — module transparent.
< 500mV	Full CCW	ka_dac_table[idx] (max inject ~2V)	brake_tables[sel][idx] (min brake)	Min brake + max anti-brake at the selected resistance.
< 500mV	CCW → CW	ka_dac_table[idx] decreasing	brake_tables[sel][idx] increasing	Brake firms while anti-brake fades out.
< 500mV	Full CW	OFF (Q1 off)	brake_tables[sel][idx] (dead short)	Maximum brake, no anti-brake.

2.4 Mode C — Keep-alive tuning (brushless)

Mode C is a setup mode for brushless motors only. The pot dials the keep-alive floor voltage live (ka_dac_table[pot_idx]) which is applied to Q1 so the driver can verify the eCom powers up correctly. When the toggle leaves Mode C, the current pot value is saved as saved_ka_dac after a 500ms debounce. This saved value is then used by Mode A as the Q1 KA floor. Return to Mode A or B for racing.

V_BLACK	Pot position	Q1 (KA preview)	Q2 (brake)	Behaviour
Mode C active, V_BLACK high	Any	OFF	OFF (suppressed)	Q1 previewing floor, pot CCW = 0V floor
Mode C active, V_BLACK low	Any	OFF	Linear per pot (brake)	Braking + KA floor preview simultaneously

V_BLACK	Pot position	Q1 (KA preview)	Q2 (brake)	Behaviour
V_BLACK > 1500mV	Any	ka_dac_table[pot_idx] live	OFF	Q1 live, eCom can verify floor
V_BLACK < 500mV	Any	OFF (Q1 off during braking)	Linear per pot	Braking, pot = configurable soft → DAC_BRAKE_HARD
Toggle away from C	Any	OFF	Normal	500ms debounce then save + LED 3×
Back in Mode A	Any	saved_ka_dac applied as floor	Linear per pot	Racing with KA floor active

During a hard brake (Q2 dead short), V_BLACK collapses toward 0V. Q1 is active trying to maintain the keep-alive floor but the low impedance of Q2's dead short will pull the floor down. D4 is essential in mode C — it blocks reverse current through Q1 when Q2 holds the track near 0V, protecting Q1's body diode from the repetitive peak current.

3. Hardware Design

3.1 Component list

Ref	Component	Value / Part	Notes
U1	MCU	STM32G051K6U6	QFN-32, 3.3V, 64MHz
U2	LDO regulator	AMS1117-3.3	WHITE rail → 3.3V, SOT-223
GD1 (LM321, ≥15V, floating supply); ZD1 BZX84C15 15V; R_Z1 10kΩ; C_Z1 100nF	Gate driver	UCC27524	Keep-alive / anti-brake FET channel (Q1), SOT-23-5
GD2	Gate driver	UCC27524	Brake FET channel (Q2), SOT-23-5
Q1	Anti-brake / keep-alive FET	IRF9Z34N P-ch	High-side switch, 18A, D2PAK
Q2	Brake FET	IRLB3034 N-ch	195A, 1.4mΩ Rds(on), D2PAK
D1	TVS diode	P6KE18CA (bidirectional)	Rail transient clamp, 18V, SMB. CA suffix essential — bidirectional variant clamps both polarity transients from inductive switching.
D2	Schottky	SS310	Q2 catch diode, 3A/100V SMB. 100V

Ref	Component	Value / Part	Notes
			reverse voltage gives margin over SS34's 40V on a 16V rail with inductive spikes.
D3	Schottky	SS310	Q1 catch diode, 3A/100V SMB. Same upgrade as D2.
D4	Schottky	SS34	Mode C series output diode — ESSENTIAL
D5	LED red	0805	Status indicator PB6
F1	Polyfuse	500mA–1A, 1812	WHITE rail supply protection for MCU/LDO only. Must NOT be in the BLACK (track) path — a polyfuse cannot handle 40A motor startup and will trip from thermal lag. The BLACK path carries full motor current; PSU overcurrent protection covers that rail.
R1	10k Ω	0402	NRST pull-up
R2	10k Ω	0402	GD2 input pull-up (safe default brake)
R3	10 Ω	0402	Q2 gate resistor
R4	10 Ω	0402	Q1 gate resistor
R5	10k Ω	0402	Pot top-end to 3.3V
R6	10k Ω	0402	PB3 reserved (no component fitted)
R7	1k Ω	0402	PB3 reserved — 1k Ω footprint, DNF
R8	15k Ω	0402	WHITE sense top divider (PA0)
R9	4k7 Ω	0402	WHITE sense bottom divider (PA0)
R10	22k Ω	0402	BLACK sense top divider (PA1)
R11	4k7 Ω	0402	BLACK sense bottom divider (PA1)
R12	330 Ω	0402	Status LED current limit
C1–C4	Decoupling	0805/0402	100nF + 10 μ F per LDO rail
C5–C7	Decoupling	0402	100nF VDD, 100nF VDDA, 4.7 μ F VDDA
J1–J3	4mm banana sockets	Track side	WHITE / BLACK / RED to track
J4–J6	4mm banana sockets	Controller side	WHITE / BLACK / RED from hand controller
J7	3-pin 2.54mm	Pot connector	3.3V / wiper / GND
J8	3-pin 2.54mm	Mode toggle	Bit0 / Bit1 / GND
J9	2-pin 2.54mm	(reserved, was capture button)	Momentary pushbutton

3.2 Pin assignments — STM32G051K6U6 QFN-32

Pin	Signal	Type	Function
PA0	WHITE sense	ADC_IN0 analog	WHITE rail ÷ R8/R9 divider. $V_{PA0} = V_{WHITE} \times 0.2386$. Safe to 16V.
PA1	BLACK sense	ADC_IN1 analog	BLACK_to_track ÷ R10/R11 divider. Motor side of D_PG.
PA3	Pot wiper	ADC_IN3 analog	10k linear pot, 0–3.3V. No centre detent.
PA4	Q2 brake gate	DAC_CH1	0–3.3V → GD2 op-amp → Q2. DAC_BRAKE_OFF... DAC_BRAKE_HARD.
PA5	Q1 KA gate	DAC_CH2	0–3.3V → GD1 op-amp (floating supply) → Q1. DAC_KA_MAX...DAC_KA_OFF.
PA9	USART1_TX	AF1	UART TX to J11 / CH340N. 115200 8N1.
PA10	USART1_RX	AF1	UART RX from J11 / CH340N.
PB0	Toggle bit 0	GPIO input, pull-up	Active LOW. Decoded with PB1: see toggle table.
PB1	Toggle bit 1	GPIO input, pull-up	Active LOW. Decoded with PB0: see toggle table.
PB3	(reserved)	GPIO	Previously capture button. Unused in Rev 11.
PB4	Voltmeter PWM	TIM3_CH1 AF1	1kHz, RC filter → J14 pin 1.
PB6	Status LED	GPIO output	Blinks 3× on keep-alive save to flash.
PB8	BOOT0	—	ROM bootloader entry (hold + NRST) for field update via USB-C/UART.

3.3 Voltage dividers

Both sense inputs use the same bottom resistor (4k7Ω) for component commonality. The top resistor differs to accommodate the different maximum voltages:

Signal	R_top	R_bot	Ratio	V_pin at 16V rail	Safe?
WHITE (PA0)	15kΩ	4k7Ω	0.2386	3.82V — marginally over	Add 12k top for 2.96V (see note)
WHITE (PA0) revised	18kΩ	4k7Ω	0.2073	3.32V	✓ safe to 3.3V rail
BLACK (PA1)	22kΩ	4k7Ω	0.1760	2.82V	✓ safe to 3.3V rail

If the track PSU can deliver up to 16V, use R8=18kΩ (not 15kΩ) for the WHITE sense divider. Update WHITE_DIV_NUM/DEN in brake_module.h to 47/234 accordingly. The original 15k value was sized for 12.4V supplies; at 16V it produces 3.82V on PA0 which exceeds the 3.3V ADC reference.

3.5 Power and thermal analysis

The following analysis confirms component ratings against the worst-case electrical demands of the application. Track PSU: 12.4V nominal, 16V maximum. Motor startup: 40A for a few milliseconds. Braking back-EMF: 20A+ momentarily. PSU cannot absorb reverse current.

Component	Worst-case demand	Rating	Verdict	Action taken
Q2 IRLB3034 (brake FET)	20A braking current continuous during brake $P = I^2 \times R_{ds} = 400 \times 0.0014 = 0.56W$ $\Delta T = 35^\circ C$ above ambient (no heatsink)	195A, 40V, $175^\circ C$ Tj $R_{ds(on)} = 1.4m\Omega$ $R_{th_ja} = 62^\circ C/W$	✓ Comfortable	No change. Motor startup current (40A) does not flow through Q2 — Q2 is OFF during throttle. Copper pour under D2PAK pad recommended.
Q1 IRF9Z34N (keep-alive FET)	Mode C keep-alive: 5 –50mA (eCom load) — negligible Simultaneous Q2+Q1: up to 46A instantaneous through Q1 body diode if D4 absent	18A, 55V, $175^\circ C$ Tj $R_{ds(on)} = 350m\Omega$ @ $V_{gs} = -3.3V$	✓ OK (with D4)	D4 mandatory in mode C. Without D4, body diode of Q1 carries 46A repetitively at 20kHz — not rated for this. D4 blocks this current path.
D1 TVS (P6KE18C A)	Motor back-EMF adds to rail: 16V + 12V peak = 28V transient Energy: $< 600W \times 1ms$ pulse capacity	600W peak (10/1000 μs), $V_c = 25.2V$ @ 1A	✓ OK	Changed to bidirectional CA variant. Unidirectional P6KE18A only clamps one polarity; inductive switching generates both.
D2/D3 SS310 (catch diodes)	Q2 turn-off spike: $L \times di/dt = 100\mu H \times 20A/1\mu s = 2000V$ unclamped D2 clamps at $V_f + V_{dc} \approx 16.4V$, energy = 20mJ	3A avg, 100V, same SMB footprint as SS34	✓ Upgraded	Changed from SS34 (40V) to SS310 (100V). D2 is essential — without it Q2's 40V Vds is instantly exceeded on turn-off. 40V reverse rating left no margin on 16V rail.
D4 MBRS360 (mode C series diode)	Repetitive ~46A pulses at 20kHz during mode C hard braking (Q1 ON-time into near-shortened track via Q2)	3A avg, 60V, 150A peak (single pulse) Same SMB footprint	✓ Upgraded	Changed from SS34 (80A peak, marginal under repetitive 20kHz stress) to MBRS360 (60V, same SMB footprint). SS54 (5A, 150A peak) is equally suitable. D4 position is the most electrically stressed diode in the design.
F1 polyfuse (WHITE rail)	MCU + LDO + gate drivers $\approx 35mA$ total on WHITE rail No motor current on	Resettable fuse, now 500mA–1A	✓ Corrected	Downrated from 25A to 500mA–1A. F1 protects the MCU/LDO supply on WHITE only. It must NOT be in the BLACK (track) path which carries

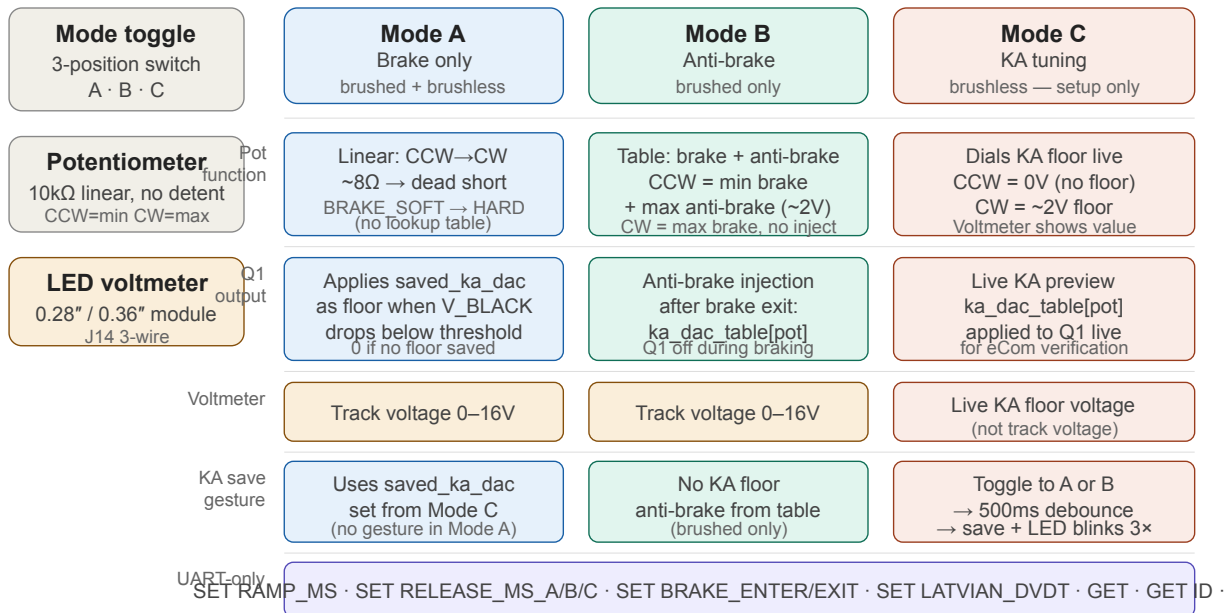
Component	Worst-case demand	Rating	Verdict	Action taken
	WHITE rail during braking or throttle			40A motor startup and 20A braking current — a polyfuse would trip.
U2 AMS1117-3.3 (LDO)	Load: $\sim 35\text{mA}$ $P = (16 - 3.3) \times 0.035 = 0.44\text{W}$ at 16V rail $\Delta T = 0.44 \times 62 = 27^\circ\text{C}$ (SOT-223, no heatsink)	1A, 125°C T_j , R_{th_ja} 62°C/W	✓ OK	$T_j = 52^\circ\text{C}$ at 25°C ambient, 97°C at 70°C ambient. Both within 125°C limit. Solder SOT-223 tab to copper pour for better thermal performance.

Motor startup current of 40A does not flow through any module component during normal operation. Q2 and Q1 are both OFF when the hand controller is delivering power ($V_{BLACK} > 1500\text{mV}$). The module is fully passive during throttle. Only during braking does current flow through Q2, and only then does the back-EMF need to be managed.

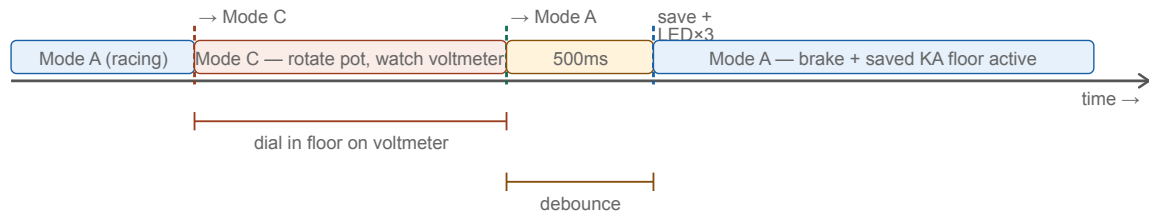
3.6 User interface — control devices

The module has three physical control devices and a voltmeter display. All are on or attached to the module enclosure. No modification to the hand controller is required.

AART Brake Module — User Interface Rev 11



Keep-alive floor setup workflow



To zero the KA floor: toggle to Mode C → rotate pot full CCW → toggle back to Mode A → wait 500ms. LED blinks when saved.
To set anti-brake without KA floor: use Mode B. Mode B does not apply the saved KA floor.

[Figure 2: User interface controls and voltmeter display logic — see AART_Brake_UI.svg]

Control	Type	Connector	Function
Mode toggle	3-position switch	PB0 (bit 0), PB1 (bit 1)	Selects Mode A (brake only, all motors — KA floor from Mode C applied if set), Mode B (brushed anti-brake, lookup table), or Mode C (brushless KA tuning, save on leaving). Read each 1ms tick.
Potentiometer	10kΩ linear rotary, no centre detent	PA3 (ADC_IN3)	Mode A: linear CCW→CW brake only (DAC_BRAKE_SOFT to DAC_BRAKE_HARD). Mode B: indexes coupled brake+anti-brake table. Mode C: dials KA floor live (CCW=0V, CW=~2V). No centre detent required.
BOOT0 button (J12)	Momentary pushbutton,	PB8/BOOT0	ROM bootloader entry for field firmware update. Not part of normal operation.

Control	Type	Connector	Function
	normally open		

3.6.1 What the potentiometer controls in each mode

The pot function depends on the mode selected by the toggle. In all modes, CCW gives minimum output and CW gives maximum:

Mode	Pot function	Output produced
Mode A — Brake only (all motors)	Linear map: CCW = selected soft min (SET BRAKE_OHMS) (~8Ω), CW = DAC_BRAKE_HARD (dead short). No lookup table.	Q2 brake only. Q1 applies saved_ka_dac as a floor independently (set via Mode C). No anti-brake in Mode A.
Mode B — Anti-brake (brushed only)	Indexes brake_dac_table[] and ka_dac_table[] simultaneously. Both brake and anti-brake live from single pot position.	CCW = min brake + max anti-brake (~2V). CW = max brake + no anti-brake. Coupled by table design.
Mode C — KA tuning (brushless, setup mode)	Dials KA floor live: CCW = DAC_KA_OFF (no floor), CW = DAC_KA_MAX (~2V). Voltmeter shows KA voltage.	Toggle away from Mode C → 500ms debounce → save saved_ka_dac + LED blinks 3×. Return to Mode A for racing.

Brake ramp-in time and trail release time are UART-only parameters (SET RAMP_MS and SET RELEASE_MS_A/B/C). They cannot be set with the pot. This keeps the pot free for the voltage setpoint function in all modes.

3.6.2 Mode C keep-alive floor procedure

Mode C keep-alive floor procedure:

1. Toggle to Mode C. The voltmeter switches from track voltage to the live KA floor voltage.
2. Rotate the pot to the desired KA floor level. CCW = 0V (no floor). CW = ~2V floor. The voltmeter tracks it live.
3. Toggle back to Mode A (or Mode B). After 500ms the floor is saved to flash. LED blinks 3×.
4. Mode A now applies the saved KA floor. Pot governs brake linearly. Car is ready for brushless racing.

What is saved and how it is applied:

Mode	What is saved	How it is applied
A	saved_ka_dac (KA floor, set by pot on leaving Mode C)	Q1 activates whenever V_BLACK drops below this voltage. 0 (DAC_KA_OFF) = no floor, pure braking. Set via Mode C.
B	No saved value in Mode B	Mode B uses the lookup table directly. Pot CCW = min brake + max anti-brake. No separate saved value.
C	N/A — Mode C is a tuning mode only	Mode C sets the value used in Mode A. It does not apply KA floor itself during racing.

All three saved values survive power cycles. They are stored in page 31 of flash (0x08007C00) and loaded at boot. The firmware defaults to DAC_KA_OFF (no injection, no floor) if no valid flash record is found.

3.7 Mini voltmeter display

A standard 3-wire mini voltmeter module (0.28" or 0.36" LED display) connects to header J14 to provide real-time voltage readout. The display serves two purposes depending on operating state:

State	Display shows	Source signal
Normal running (Mode A or B)	Track supply voltage (WHITE rail), 0–16V	PB4 PWM duty proportional to ADC-sampled V_WHITE
Mode A or B running (any time)	Track supply voltage (WHITE rail), 0–16V	PB4 PWM duty proportional to live pot reading
Mode C active (KA tuning)	Live keep-alive floor voltage, 0–2V	PB4 PWM duty proportional to live pot reading

3.7.1 J14 connector pinout

Pin	Signal	Connect to voltmeter
1	Vin — signal input. 0–16V in Mode A/B (track voltage). 0–2V in Mode C (KA floor).	Signal / measurement input wire
2	GND	Negative / GND wire
3	Vcc +3.3V	Power supply wire

The voltmeter shows two distinct voltage ranges depending on mode. In Mode A and B (normal running), the PWM duty is scaled so the display reads the actual WHITE rail voltage directly: 0–16V+ (the voltmeter Vin input accepts up to 30V). In Mode C, the PWM output is

scaled so the display reads 0–2V, corresponding directly to the keep-alive floor voltage being dialled in. This means the display is calibrated to show meaningful physical voltages in both contexts, not a proportional fraction of 3.3V.

3.4 Safe default behaviour

On power loss or MCU fault, the brake FET (Q2) must default to ON (hard brake). This is achieved by a pull-up resistor on the Q2 op-amp buffer input. When the MCU is unpowered, the buffer output rises to the braking region and Q2 conducts.

- The pull-up circuit holds the Q2 gate driver input at a voltage that places Q2 in its linear braking region (~1.8V target). The car brakes at moderate force — not a violent dead short, but not free-rolling.
- `brake_force_safe()` writes directly to the DAC registers and is called from `hard_fault_handler` and `nmi_handler`.
- The hold-up capacitor (C_HU 100µF on 3.3V rail) maintains MCU power for ~300ms after PSU cut, allowing the firmware to maintain the full programmed brake characteristic through power-off.

4. Firmware Architecture

4.1 File structure

File	Description
Core/Inc/ brake_module.h	Constants, pin map, mode/state enums, BrakeCtx_t, public API. Rev 9.
Core/Src/ brake_module.c	Full control logic: clock/GPIO/DAC/ADC/DMA/voltmeter setup, flash load/save, mode state machine, brake_tick().
Core/Inc/ uart_debug.h	DebugParams_t struct, uart_debug public API.
Core/Src/ uart_debug.c	UART telemetry, SET/GET command parser, flash SAVE.
Core/Src/main.c	Top-level: SysTick ISR, fault handlers, main loop calling brake_tick().
stm32g051k6.ld	Linker script: 32K flash at 0x08000000, 18K RAM at 0x20000000. Page 31 reserved for NV storage.
Makefile	Docker build: arm-none-eabi-gcc, -DSTM32G051, libopenm3 linked.

4.2 Timing and scheduling

The module uses a polling architecture. `brake_tick()` is called from the main loop as fast as possible. A millisecond counter incremented by SysTick ISR provides internal pacing. ADC

conversions run via DMA so the CPU never blocks waiting for a result. All control decisions happen in the 1ms tick context.

4.3 ADC configuration

ADC1 is configured in scan mode with three channels in sequence:

- Rank 1 — PA0 (CH0): WHITE rail voltage sense (R8/R9 divider)
- Rank 2 — PA1 (CH1): BLACK_to_track voltage sense (R10/R11 divider, motor side of D_PG)
- Rank 3 — PA3 (CH3): pot wiper (10k linear, 0–3.3V)

Results are transferred to a 3-element DMA buffer. DMA TC interrupt sets `s_adc_ready` flag. `brake_tick()` reads the flag and processes one set of readings per call. Trigger: software (SWSTART), retrigged at end of each DMA transfer.

4.4 DAC configuration

DAC1 channels 1 and 2 are enabled simultaneously:

- DAC_CH1 (PA4) — Q2 brake gate. 0–3.3V linear, maps from the selected minimum resistance (2/3/4/6/8Ω) to dead short at the FET.
- DAC_CH2 (PA5) — Q1 KA/anti-brake gate. 0–3.3V, where 3.3V = Q1 off, ~2.0V = max conduction.

Both channels use software trigger mode. `brake_tick()` calls `set_brake_dac()` and `set_ka_dac()` which write to DAC_DHR12R1/R2 and fire DAC_SWTRIGR. The op-amp buffers (GD1, GD2) buffer the DAC output to drive the FET gates in linear mode.

4.5 Key constants

Constant	Value	Description
DAC_BRAKE_OFF	0	Q2 off. 0V gate, $V_{gs} < V_{th}$.
DAC_BRAKE_SOFT	1860	Legacy reference (~1.5V gate, ~8Ω). Superseded by SET BRAKE_OHMS table selection.
DAC_BRAKE_HARD	3095	Q2 near dead short (~2.5V gate).
DAC_KA_OFF	4095	Q1 off (3.3V gate, $V_{gs} \approx 0$ relative to source).
DAC_KA_MAX	2480	Q1 maximum conduction (~2.0V gate, ~2V injection).
DAC_BRAKE_SOFT	1860 (DAC_BRAKE_SOFT)	Minimum brake resistance is selectable via SET BRAKE_OHMS (snaps to 2/3/4/6/8Ω), choosing one of five stored tables. <code>brake_ohms_idx</code> (0-4) stored in flash.

Constant	Value	Description
		Default index 1 (~3Ω).
DAC_KA_MAX (anti-brake max)	2480 (DAC_KA_MAX)	Maximum Q1 conduction. Anti-brake level follows pot via ka_dac_table[].
KA_SAVE_DEBOUNCE_MS	5000	KA save debounce after leaving Mode C. Fixed at 500ms in firmware.
RAMP_MS_MAX	200	Maximum ramp-in time. SET RAMP_MS 0–200.
RELEASE_MS_MAX	500	Maximum trail release time. SET RELEASE_MS_A/B/C 0–500.
FLASH_MAGIC	0xAA270008	Rev 11. Old records invalidated on first boot.
FW_ID_STRING	AART-BRAKE-v11-STM32G051	Returned by GET ID.

4.6 Control flow

brake_tick() runs the full state machine each call. The flow:

- 1. Read ADC DMA buffer — white_mv, black_mv, pot_raw.
- 2. Read toggle. Detect mode transitions — specifically leaving Mode C (triggers 500ms debounce save timer).
- 3. Latvian brake check — WHITE dV/dt. If triggered, apply DAC_BRAKE_HARD and return.
- 4. Update brake_active via hysteresis (BRAKE_ENTER/EXIT_MV thresholds on BLACK_to_track).
- 5. Dispatch to mode handler: Mode A (linear brake + KA floor), Mode B (anti-brake table), or Mode C (KA tuning).
- 6. Mode A: linear map pot → Q2 brake, saved_ka_dac applied as Q1 floor when V_BLACK low. Mode B: table lookup for both outputs. Mode C: live ka_dac_table preview on voltmeter, linear brake during braking.
- 7. If leaving Mode C and 500ms has elapsed: save ka_dac_table[pot_idx] as saved_ka_dac, blink LED 3×.
- 8. Update voltmeter PWM: track voltage normally; live ka_dac_table value while in Mode C.
- 8. Update LED blink state.

6. Bench Testing

6.1 Brake detection verification

Inject a variable DC voltage on the BLACK_to_track node via a bench supply or potentiometer. Observe UART telemetry state field:

- $V_{\text{BLACK}} < 500\text{mV}$ → state transitions to BRAKING (or RAMP_IN if ramp enabled)
- $V_{\text{BLACK}} > 1500\text{mV}$ → state returns to PASS
- Between 500mV and 1500mV → state holds (hysteresis prevents chatter)

6.2 Power budget

Production target at 4MHz LP Run mode: approximately 522 μA total. MCU ~372 μA , op-amp buffers ~50 μA each, LDO (MCP1700) 2 μA quiescent. Hold-up capacitor C_HU 100 μF sustains MCU for >300ms after PSU cut. See section 7A.6 for full analysis.

7. Open Items and Known Limitations

Item	Description	Status
Q1 turn-off gate drive	IRF9Z34N source is at WHITE rail (12–16V). Q1 turn-off requires the gate near V_WHITE. Resolved by the GD1 floating supply (section 7A.7): ZD1 (15V zener) + R_Z1 + C_Z1 reference GD1 to the WHITE rail so its output swings WHITE–15V to WHITE. DAC_KA_OFF = 4095 now turns Q1 off correctly. GD1 is an LM321 ($\geq 15\text{V}$ supply). Bench-verify turn-off at 12V and 16V.	Resolved — GD1 floating supply added (section 7A.7). ZD1 (BZX84C15 15V zener) + R_Z1 (10k Ω) + C_Z1 (100nF) reference GD1 (LM321) to the WHITE rail; output swings WHITE–15V (Q1 on) to WHITE (Q1 off). DAC_KA_OFF = 4095 turns Q1 off correctly. Remaining: bench-verify turn-off margin at 12V rail (marginal but only needed in passthrough, not racing).
D_PG CTRL pin polarity	LTC4412 CTRL pin polarity — verify active-low vs active-high from datasheet before PCB layout.	Open — bench verify.
Latvian brake primary trigger	WHITE dV/dt detects PSU cut and car-out-of-slot events. Implemented in current firmware (STATE_LATVIAN_BRAKE). Threshold configurable via SET_LATVIAN_DVDT. Disabled by default (0 = off). The alternative trigger (BLACK braid loss) was considered but WHITE dV/dt	Implemented. UART-configurable threshold (SET_LATVIAN_DVDT). Closed.

Item	Description	Status
	covers both PSU cut and out-of-slot and is sufficient.	

7A. Production Architecture — Design Decisions and Analysis

7A.1 Overview

This section documents the key architectural decisions made during the design process, the alternatives considered, and the rationale for each choice.

7A.2 MOSFET pass gate — gate drive options

The BLACK controller input path requires a near-zero-drop pass gate that the MCU can open to isolate the controller during brake takeover. Three options were evaluated:

Option	Circuit	Pros	Cons	Decision
A	NPN level-shift (2 transistors)	Simple, cheap, bench-verifiable	Voltage drop at high current; NPN saturation heating	Prototype only
B	P-channel MOSFET + NPN driver	Low drop; standard circuit	Gate drive complexity; body diode direction	Intermediate
C (chosen)	LTC4412 ideal diode IC	Near-zero drop both directions; CTRL pin override; 20–50 μ A quiescent; auto gate management	Cost (~€0.80/unit); 8-pin SOT package	Production

7A.3 Linear mode FET control

Both Q1 and Q2 operate in their linear (triode) region rather than as switches. This eliminates PWM switching noise on the braid, which is critical for brushless eCom compatibility. It also removes 20kHz conducted emissions that would complicate CE marking. See section 11.0 for full analysis.

7A.4 Hold-up capacitor sizing

A 100 μ F SMD electrolytic on the 3.3V rail provides ~300ms MCU hold-up after PSU cut at 522 μ A production load. This ensures the firmware maintains the full programmed brake through the power-off transient. See section 7A.6 for worked calculation.

7A.5 Production MCU: STM32G051K6U6

- QFN-32 — provides all required GPIO and DAC peripherals.
- 12-bit DAC (PA4/PA5) — required for linear gate voltage control.
- 32KB flash, 18KB RAM, 64MHz Cortex-M0+ — adequate for Rev 9 firmware.
- Available at JLCPCB. Not a special-order part.

7A.6 Power-down, brownout and hold-up capacitor

7A.6.1 The problem: braking must survive PSU power-off

A conventional brake resistor continues to brake the car when the PSU cuts because it requires no power. An active module loses MCU power at PSU cut. The pull-up on GD2 provides a hardware safe default (Q2 enters braking region), but the hold-up capacitor extends firmware-controlled braking through the transient.

7A.6.2 STM32G051 power supply supervisor: POR, PDR and BOR

Circuit	Function	Threshold	Behaviour on VDD falling
POR (Power-On Reset)	Holds MCU in reset during power-up	VPOR $\approx$ 1.7V (rising)	Not relevant on falling edge
PDR (Power-Down Reset)	Resets MCU when VDD falls below minimum	VPDR $\approx$ 1.6V (falling)	MCU reset. DAC outputs collapse.
BOR (Brownout Reset)	Programmable earlier warning threshold via option bytes	VBOR1–VBOR4: 1.7V–2.8V selectable	MCU reset when VDD < VBORx

7A.6.3 Why the STM32G051 has suitable power consumption

Component	Prototype (64MHz)	Production (4MHz)	Notes
STM32G051 MCU	~6mA	~372 μ A	G0 family: ~93 μ A/MHz. 4MHz $\times$ 93 = 372 μ A.

Component	Prototype (64MHz)	Production (4MHz)	Notes
Gate drivers GD1/GD2	~10mA (UCC27524)	~50μA (op-amp)	Dominant saving. Linear mode op-amps replace switching drivers.
LDO	~5mA (AMS1117)	~2μA (MCP1700)	AMS1117 quiescent is 5mA. MCP1700 is 2μA.
ADC, DAC, GPIO	~500μA	~100μA	ADC scanning 3 channels at 1ms interval.
Total	~21.5mA	~524μA	Production well under 1mA.

7A.6.4 Hold-up capacitor sizing — worked calculation

$C = I \times t / \Delta V$, where $I = 524\mu A$, $t = 100ms$, $\Delta V = 3.3 - 1.7 = 1.6V$:

- $C = 0.000524 \times 0.1 / 1.6 = 32.8\mu F$ minimum
- Use 100μF (3× margin). Verification: $t = 100 \times 10^{-6} \times 1.6 / 0.000524 = 305ms$.
- 3× margin accounts for X5R/X7R capacitance derating at DC bias (~40% typical), age degradation, and load estimate uncertainty.

7A.6.5 Component selection: 100μF 6.3V SMD electrolytic

Parameter	Value	Rationale
Capacitance	100μF	>300ms hold-up at 524μA, 3× margin
Voltage rating	6.3V min, 10V preferred	Reduces derating on 3.3V rail
Package	SMD electrolytic or 1210 polymer	Polymer SMD has lower ESR
Stored energy	$E = \frac{1}{2}CV^2 = 0.54mJ$	Powers MCU only. Well below any regulatory stored-energy threshold.

The 100μF hold-up cap is in addition to the ST-recommended 100nF + 4.7μF decoupling at VDD pins.

7A.6.6 Behaviour on power-down

Time	Event	MCU state	Brake output
$t = 0$	PSU switched off. WHITE rail begins to fall.	Running normally	Programmed brake at pot-set level
$t \approx 0$ –10ms	LDO ceases to regulate. C_HU	Running from capacitor	DAC maintains programmed brake

Time	Event	MCU state	Brake output
	supplies VDD.		
$t \approx 0\text{--}305\text{ms}$	VDD falls from 3.3V toward 1.7V as C_HU discharges.	Running. ADC less accurate below 2V.	Programmed brake maintained
$t \approx 305\text{ms}$	VDD < PDR threshold. MCU resets. DAC collapses.	Reset	Pull-up circuit takes over
$t > 305\text{ms}$	R2 pull-up holds Q2 gate in braking region.	No power	Hardware safe-default brake

7A.7 Q1 gate drive — floating supply circuit

Q1 (IRF9Z34N, P-channel) has its source at the WHITE rail (12–16V). To control Q1 in linear mode the gate must swing from near WHITE (Q1 off, $V_{gs} \approx 0\text{V}$) down to $\approx 2\text{--}4\text{V}$ (Q1 on, $V_{gs} \approx -8$ to -10V). The op-amp buffer GD1, if powered from 3.3V, can only output 0–3.3V, leaving Q1 in deep enhancement even at maximum DAC output. This is resolved with a floating supply that references GD1 to the WHITE rail rather than to GND.

7A.7.1 Circuit

A 15V zener (ZD1) sits between the WHITE rail and GD1's positive supply pin, with a current-limiting resistor R_Z1 from WHITE. GD1's negative supply connects to the WHITE–15V node. GD1's rails then float with WHITE:

- GD1 V+ = WHITE (12–16V). GD1 V– = WHITE – 15V (–3V to +1V at 12–16V rail).
- GD1 output range: (WHITE–15V) to WHITE. Gate at WHITE $\rightarrow V_{gs} = 0\text{V} \rightarrow$ Q1 off. Gate at WHITE–15V $\rightarrow V_{gs} = -15\text{V} \rightarrow$ Q1 fully on.

DAC2 (0–3.3V, GND-referenced) drives GD1's input; a rail-to-rail input op-amp accepts this common-mode range. The firmware constant DAC_KA_OFF = 4095 (3.3V) now correctly drives the gate to near WHITE and turns Q1 off. No firmware changes are required.

7A.7.2 New components

- ZD1 — BZX84C15 (15V zener, SOT-23). Cathode to WHITE, anode to GD1 V+ and C_Z1. At 16V rail, zener current $\approx (16-15)/10\text{k} = 0.1\text{mA}$, $\sim 0.1\text{mW}$ dissipation.
- R_Z1 — 10k Ω 0402. Series from WHITE to ZD1 cathode, sets zener current.
- C_Z1 — 100nF 0402. Decoupling across GD1's floating supply, placed close to GD1.
- GD1 — must be rated for $\geq 15\text{V}$ supply. LM321 (SOT-23-5, 32V, rail-to-rail output, JLCPCB basic) is the chosen part. SOT-23-5 5–6V op-amps (LMV321, MCP6001) are NOT suitable.

Note: At a 12V rail the floating supply margin is reduced and Q1 turn-off is marginal; this is acceptable because full Q1 turn-off is only needed during Mode A passthrough, not during racing. Bench-verify turn-off at both 12V and 16V before PCB commit. D4 (MBRS360) in series with Q1's output remains essential whenever Mode A runs with a non-zero KA floor.

7A.8 FET thermal analysis — are heatsinks required?

Both Q1 and Q2 are in D2PAK packages whose copper tab acts as the primary thermal path. The question is whether the PCB copper pour is sufficient or whether additional heatsinking is needed.

7A.8.1 Q2 (IRLB3034, N-channel brake FET)

Q2 operates in linear mode as a variable resistance across the motor terminals. The worst-case dissipation is at minimum brake (soft end of pot travel, $R_{ds} \approx 8\Omega$), not at dead short. At dead short the FET is fully enhanced ($R_{ds} \approx 1.4m\Omega$) and dissipates negligible power. The 8Ω figure below is the gentlest selectable setting and so represents the worst case across all SET BRAKE_OHMS values; firmer settings dissipate less.

Worst-case calculation at soft brake:

- Motor back-EMF at racing speed $\approx$ track voltage $\approx 12V$
- Motor winding resistance $\approx 2\text{--}5\Omega$ (typical 1/32 brushed motor)
- Total series resistance: $R_{\text{motor}} + R_{\text{Q2}} = 3\Omega + 8\Omega = 11\Omega$
- Braking current $I = 12V / 11\Omega = 1.1A$
- Power dissipated in Q2: $P = I^2 \times R_{\text{Q2}} = 1.1^2 \times 8 = 9.7W$ peak

9.7W peak sounds alarming but braking is intermittent. A slot car brakes for approximately 0.1–0.3 seconds per corner, at a duty cycle of roughly 10–20%. Average power over a lap: $9.7W \times 0.15 = 1.5W$ average.

D2PAK thermal parameters:

- θ_{jc} (junction to case) $\approx 1.2^\circ C/W$
- θ_{ca} (case to ambient, D2PAK tab soldered to PCB copper pour, no heatsink) $\approx 20\text{--}30^\circ C/W$ with a 20×20mm copper pour
- Peak junction temperature during 0.3s brake event: $T_j \approx 25 + (1.5 \times 30) + (9.7 \times 1.2) \approx 25 + 45 + 12 = 82^\circ C$. Well within $150^\circ C$ maximum.

Conclusion: no heatsink required for Q2. A 20×20mm 1oz copper pour under the D2PAK tab on both top and bottom copper layers, stitched with thermal vias, is sufficient. The tab must be fully soldered — a dry or cold joint under the tab dramatically raises θ_{ca} .

7A.8.2 Q1 (IRF9Z34N, P-channel KA/anti-brake FET)

Q1 carries eCom supply current (10–50mA typical) and anti-brake injection current (peak $\approx 500mA$, very brief). Worst-case dissipation:

- $P = I^2 \times R_{ds(on)} = 0.5^2 \times 0.35\Omega = 87mW$

Negligible. No heatsink required for Q1. Normal PCB land pattern for D2PAK is sufficient.

7A.9 USB-C interface — UART bridge and auxiliary power

A USB-C connector is added to provide two functions: a UART terminal connection to a laptop or PC for issuing SET/GET commands and monitoring telemetry, and auxiliary power to the module when the track is not connected (bench configuration, parameter setting before a race).

7A.9.1 New components

- J_USB — USB-C receptacle, 16-pin SMD (HRO TYPE-C-31-M-12 or equivalent, JLCPCB). Carries VBUS, GND, D+, D-, CC1, CC2. Mid-mount or top-mount.
- U3 — CH340N USB-UART bridge, SOP-8 (JLCPCB C506813, ~€0.15). Internal 12MHz clock, no crystal required. Supports 3.3V supply.
- R_CC1, R_CC2 — 5.1kΩ 0402 each, CC1 and CC2 to GND. Required for USB-C DFP detection (signals to host that this is a UFP device and requests 5V/0.9A).
- D_USB — BAT54 Schottky diode, SOT-23 (or equivalent, JLCPCB). VBUS in to LDO input, prevents reverse current from track WHITE rail reaching the USB host.
- F_USB — 500mA polyfuse 0805, in VBUS line before D_USB. Protects USB host from shorts on the module.
- C_USB1 — 10μF 10V MLCC 0805, VBUS decoupling.
- C_USB2 — 100nF 0402, CH340N VCC decoupling.

7A.9.2 Power supply arrangement

The module has two power inputs: the track WHITE rail (12–16V) and USB VBUS (5V). Both feed the existing LDO (U2) input through protection diodes:

- WHITE rail → F1 (polyfuse, existing) → D_WHITE (SS310 Schottky, existing) → LDO input
- VBUS → F_USB (polyfuse, new) → D_USB (BAT54 Schottky, new) → LDO input

Both diodes OR into the LDO input. The higher voltage dominates: WHITE (12–16V) always has higher voltage than VBUS (5V) so WHITE powers the LDO when both are connected. The LDO output is 3.3V in both cases.

USB host protection: D_USB blocks the WHITE rail voltage from reaching the USB host. The USB host sees only its own VBUS through F_USB. There is no path from WHITE to VBUS. This is the critical isolation requirement and it is met by the series Schottky.

Note: When only USB is connected (no track), VBUS supplies the LDO at 5V. The LDO regulates to 3.3V normally. The module is fully functional for UART configuration. The FET gate drivers and FETs are not powered (WHITE = 0V) but the MCU and UART run normally.

7A.9.3 CH340N wiring

The CH340N is configured for 3.3V operation (V3 pin shorted to VCC, both at 3.3V):

- VCC (pin 8) → 3.3V rail

- V3 (pin 1) → VCC (shorted, enables 3.3V I/O mode)
- GND (pin 2) → GND
- D+ (pin 3) → USB-C D+
- D- (pin 4) → USB-C D-
- RXD (pin 5) → MCU PA9 (USART1_TX). CH340N receives what MCU transmits.
- TXD (pin 6) → MCU PA10 (USART1_RX). CH340N transmits to MCU.
- Unused pins (7, 8) left unconnected or tied per datasheet

Note: CH340N batch numbers > B40 include internal backflow prevention. For earlier batches, a series Schottky on RXD prevents current injection into the MCU when the MCU is unpowered but USB is connected.

7A.9.4 USB-C connector wiring

- VBUS (pins A4, B4) → F_USB → D_USB → LDO input
- GND (pins A1, B1, A12, B12) → GND
- D+ (pins A6, B6) → CH340N pin 3
- D- (pins A7, B7) → CH340N pin 4
- CC1 (pin A5) → R_CC1 (5.1kΩ) → GND
- CC2 (pin B5) → R_CC2 (5.1kΩ) → GND
- SBU1, SBU2, SSTX, SSRX: not connected (USB 2.0 only)

7A.9.5 Driver installation

The CH340N requires a driver on the host computer. On Linux and macOS (10.9+) the ch341 kernel module is included and loads automatically. On Windows 10/11 the driver is available from the WCH website (www.wch-ic.com). Once installed the module appears as a standard COM port (Windows) or /dev/ttyUSBx (Linux) or /dev/tty.wchusb* (macOS). Any serial terminal at 115200 8N1 can then issue UART commands.

7A.10 Configurable minimum brake resistance

The minimum braking resistance (the resistance at pot CCW, the gentlest brake setting) is motor-dependent. A small 1105 brushless motor needs roughly 3Ω, while a Group 12 brushed motor may want roughly 5Ω. The original hardcoded ~8Ω (DAC_BRAKE_SOFT = 1860) suited neither well. This value is now selectable at runtime without being hardware-dependent.

7A.10.1 Why stored tables, not runtime scaling

Two implementations were considered. The first rescaled a single base table on every control tick to move its soft endpoint. The second — adopted here — stores five precomputed tables, one per nominal resistance, and selects one by index. The stored-table approach was chosen because it performs zero arithmetic on the table in the control loop: the 1ms tick is a pure array lookup, keeping timing fully deterministic and avoiding any floating-point or division on the Cortex-M0+ (which has no FPU).

7A.10.2 The five tables

Five brake tables are stored, each a gamma-1.8 curve from its soft endpoint up to DAC_BRAKE_HARD (dead short). The resistances are spaced non-linearly (2, 3, 4, 6, 8Ω — roughly a $\sqrt{2}$ progression) so the steps are finer at the firm/low-ohm end where small changes matter most, and coarser at the gentle end. Higher DAC = higher Q2 gate voltage = lower Rds(on) = lower resistance, so lower-ohm settings use higher soft endpoints:

- Index 0 — ~2Ω full scale (soft DAC 2300)
- Index 1 — ~3Ω full scale (soft DAC 2200) — default; suits 1105 brushless
- Index 2 — ~4Ω full scale (soft DAC 2100)
- Index 3 — ~6Ω full scale (soft DAC 1950)
- Index 4 — ~8Ω full scale (soft DAC 1860) — original default; gentle

Mode B (anti-brake) indexes the selected table directly. Mode A and Mode C (linear maps) use the selected table's soft endpoint as the CCW value of map_u16. The anti-brake (ka_dac_table) is unaffected — it is a voltage injection, independent of brake resistance.

7A.10.3 UART command and storage

- SET BRAKE_OHMS <ohm> takes the requested resistance and snaps to the nearest stored table (2/3/4/6/8Ω). Takes effect immediately; SAVE persists it.
- The selected index (0-4) is stored in flash DW2 (replacing the former brake_soft_dac slot).
- GET reports the current setting as brake_ohms=N.
- Default on first boot or after flash erase: index 1 (~3Ω).

7A.10.4 Calibration

Note: The 1-5Ω labels are nominal estimates. True effective resistance at a given gate voltage depends on the specific FET, its temperature, and the instantaneous braking current, and cannot be computed in firmware. The soft endpoints (2300/2200/2100/1950/1860 for 2/3/4/6/8Ω) are starting points only.

To calibrate: select each setting with SET BRAKE_OHMS, measure or feel the actual braking behaviour with the target motor under load, and if the endpoints need adjustment, edit OHM_LABELS / SOFT_ENDPOINTS in tools/gen_brake_tables.py, regenerate the tables, and rebuild the firmware. This keeps the runtime path free of any table arithmetic while allowing precise per-motor tuning.

7A.9.6 Firmware update over USB-C

No custom bootloader is required to update firmware over the USB-C connection. The STM32G051 contains a factory bootloader in system ROM that cannot be erased and supports firmware loading over USART1 (PA9/PA10) — the same pins the CH340N bridge is wired to. A single USB-C cable to a host PC is therefore sufficient to reflash the module.

Important distinction: this is the ROM UART bootloader reached through the USB-to-UART bridge, not native USB DFU. The STM32G051 has no USB device peripheral (USB appears only on the larger G0B1/G0C1 parts), so the module does not enumerate as a DFU device. From the host side the workflow is identical to any serial flash: open the CH340N COM port and use stm32flash or STM32CubeProgrammer in UART mode.

7A.9.7 CH340N reset limitation and update procedure

The CH340N is the minimal 8-pin variant and does not expose the DTR/RTS modem-control lines that auto-reset circuits use. The bootloader entry therefore cannot be automated from the flashing tool; it is performed manually with the BOOT0 (J12) and NRST (J13) buttons already present in the design:

- Connect the USB-C cable to the host PC. The module appears as a COM port (/dev/ttyUSB* on Linux, /dev/tty.wchusb* on macOS).
- Hold BOOT0 (J12) high, press and release NRST (J13), then release BOOT0. The MCU starts in the ROM bootloader, listening on USART1.
- On the host, flash the image: `stm32flash -w aart-brake.bin -v /dev/ttyUSB0` (or STM32CubeProgrammer in UART mode). Baud is auto-detected from the 0x7F sync byte.
- Press and release NRST again (BOOT0 now low). The new firmware runs.

Note: If automatic, button-free updates are ever required, a small application bootloader could be added that is triggered by a UART command (e.g. ENTER_DFU), receives the new image, and jumps to it. This costs flash, a partitioned memory map, and the usual brick risk of a self-update path, so it is not used here — the ROM UART bootloader with manual BOOT0/NRST is the lower-risk standard route for occasional updates.

Note: Before PCB commit, confirm against ST application note AN2606 that the ROM bootloader USART maps to PA9/PA10 on the specific STM32G051 variant used. This holds for the G0x1 family but is worth a quick verification, as a mismatch would prevent the CH340N bridge from reaching the bootloader.

7A.10.5 How brake and anti-brake are encoded in the table

In Mode B a single pot position drives two outputs simultaneously. The coupling is purely positional: the pot reading is reduced to one 8-bit index, and that index reads two parallel 256-entry tables at the same time — one for the brake FET (Q2), one for the anti-brake / keep-alive FET (Q1).

The decode runs every 1ms tick and contains no arithmetic or interpolation:

```
idx      = pot_raw >> 4;                // 12-bit ADC -> 8-bit index
brk_dac  = brake_tables[active_ohms_idx()][idx]; // Q2 gate drive
ka_dac   = ka_dac_table[idx];           // Q1 gate drive
```

Both stored values are 12-bit DAC codes (0–4095), i.e. gate-drive levels that become gate voltages through the op-amp buffers. They are not ohms or volts directly — the mapping from a DAC code to an actual resistance or injection voltage is FET-, temperature- and current-dependent and must be calibrated on the bench.

The two tables are built to sweep in opposite directions, so rotating the pot trades one output for the other:

- Index 0 (pot full CCW): brake at its soft minimum, anti-brake at maximum (ka_dac ≈ 2480, ~2V forward injection).
- Index 255 (pot full CW): brake at dead short (3095), anti-brake off (ka_dac = DAC_KA_OFF = 4095, Q1 fully off).
- Higher brake DAC = higher Q2 gate voltage = lower R_{ds} = more braking. Lower anti-brake DAC = more Q1 conduction = more injection. So as the brake firms up, the anti-brake fades out.

The brake curve uses a gamma-1.8 shape giving fine resolution at the soft end; the anti-brake curve fades to zero before full CW (by default at about 65% of pot travel) so the firmest part of the brake range carries no injection at all.

7A.10.6 Are separate tables per resistance necessary?

The five stored brake tables currently share the same gamma shape and differ only in where their soft (CCW) endpoint sits. Strictly, that is redundant: the shape is one table and the per-resistance part is just five endpoint values. Two factorings are valid:

- Five full tables (current design): ~2.5KB of flash (brake 2560 B + shared anti-brake 512 B + labels/endpoints 20 B ≈ 3.1KB). Buys zero runtime math and the freedom to give each resistance a different curve shape later.
- One shape + five endpoints, scaled at runtime: ~512 B plus ten endpoint values, at the cost of one multiply and one divide per tick in Mode B. Trivial on the M0+, but it reintroduces the runtime table arithmetic the stored-table design was chosen to avoid.

Because flash is not tight (32KB total, ~12KB used) and there is a plausible future need to give the low-ohm brushless profiles a distinct brake or anti-brake feel, the five-table form is retained. The redundancy is only redundancy while every profile keeps the same shape — the moment any profile diverges (a sharper brake curve, an earlier anti-brake fade), the separate tables stop being redundant. If all profiles are guaranteed to stay identical in shape, the single-shape-plus-endpoints form is the more honest choice.

Per-profile anti-brake tables are also supported (one `ka_dac_table` per resistance instead of a shared one), raising the cost to about 5.1KB — only worth it if the injection profile must differ by motor type.

7A.10.7 Table generation and inspection tooling

Two Python tools accompany the firmware (in `firmware/tools/`). They are the authoritative way to regenerate the tables after bench calibration:

- `gen_brake_tables.py` — builds the C source (`brake_ohms_label[]`, `brake_soft_dac_by_ohm[]`, `brake_tables[][]` and the anti-brake table). Each resistance is described by a Profile with independent knobs for both curves: `brake_soft` and `brake_gamma` shape the brake sweep; `ka_max`, `ka_gamma` and `ka_zero_at` shape the anti-brake sweep and set where injection fades to zero. Run with `--per-profile-ka` to emit independent anti-brake tables. The script prints the exact flash cost of the chosen configuration.
- `test_brake_tables.py` — reproduces the firmware decode (`pot >> 4 -> index -> two lookups`) and reports the brake and anti-brake DAC values at any pot setting, with rough physical estimates. Use `--ohms N --pot P` for a single point or `--sweep` for the full travel.

Calibration workflow: select a profile, measure the real braking and injection behaviour with the target motor under load, adjust that profile's endpoints or gamma in `gen_brake_tables.py`, regenerate, and rebuild. This keeps the runtime path a pure lookup while allowing precise per-motor tuning.

Note: The ohm and injection-voltage figures printed by `test_brake_tables.py` are interpolations from hand-set anchor points, not measurements. The DAC codes are exact; the physical readouts are orientation only until calibrated on the bench.

7B. Key Design Decisions

Decision	Chosen approach	Alternative considered	Rationale
Q2 safe default behaviour	Pull-up on Q2 gate driver input drives Q2 into braking region when MCU off	No safe default; separate fail-safe circuit	Pull-up voltage targets ~1.8V (lower linear region). Car brakes but not violently. Essential safety requirement.
Keep-alive suppression	Q1 off when <code>V_BLACK > floor</code> voltage derived from <code>saved_ka_dac</code> and <code>white_mv</code>	Continuous injection regardless of <code>V_BLACK</code>	Only floor below setpoint; no interference with controller above it.

Decision	Chosen approach	Alternative considered	Rationale
Ramp shape (entry)	Exponential (geometric decay: $\text{brake_dac} += \text{error} \times \alpha/256$ per ms)	Linear ramp	Fast initial bite then gentle tail. Matches driver expectation on hard braking.
Anti-brake ceiling	2V (DAC_KA_MAX=2480, ~2V at 12–16V rail)	Higher voltage	2V is top of practical range for commercial controllers (0.2–2V typical). Set via Mode B lookup table — pot position determines anti-brake level at all times.
KA floor save timing	Mode C save on toggle-out after 500ms debounce	Immediate pot takeover	500ms debounce prevents accidental saves from toggle contact bounce. No pot handback needed — Mode A pot is always live.
(reserved, was capture button)	No capture button in Rev 11. KA floor saved by toggle gesture (leaving Mode C).	ADC pin (original design, before PA4/PA5 were reassigned to DAC outputs)	Toggle-out is the natural gesture for mode change. No separate button needed. PB3 reserved for future use.
Flash storage	Four 64-bit double-words at page 31 (Rev 11). Magic 0xAA270008.	EEPROM emulation	G0 flash requires 64-bit aligned writes. Page 31 reserved by linker. Four DWs store all tuneable parameters.
MCU package	STM32G051K6U6 QFN-32	G031 SOIC-8	QFN-32 provides PA4/PA5 DAC outputs, PB0/PB1/PB3/PB4 GPIO, sufficient for all signals. G051 has 12-bit DAC required for linear mode.
Toggle positions	3-position: A (brake all motors) / B (brushed anti-brake) / C (KA tuning)	2-position or mode-less	Mode A is the racing mode for all motor types (brushed and brushless). Mode B adds coupled anti-brake for brushed. Mode C is KA tuning only — set the floor and return to Mode A.
Minimum brake DAC	Minimum brake resistance selectable via SET BRAKE_OHMS, snapping to one of five non-linearly spaced stored tables (2/3/4/6/8Ω). No runtime table math.	Hardcoded minimum	Different motors need different minimum braking resistance (e.g. ~3Ω for a 1105 brushless, ~5Ω for Group 12 brushed). Storing five tables avoids any per-tick table computation while keeping the choice field-selectable.

8. Firmware Build — libopencm3 and Docker Toolchain

8.1 Library choice

libopencm3 is used instead of STM32 HAL. It provides direct register-level access with thin C wrappers, no RTOS, no hidden interrupt overhead, and a deterministic build. The firmware has no dynamic memory allocation and no C++ dependencies.

8.2 G0-specific differences from other STM32 families

- Flash programming requires 64-bit double-word writes (`flash_program_double_word`). 32-bit writes are not supported on G0.
- DMA requests are routed via DMAMUX1. ADC1 uses DMAMUX request 5.
- DAC1 has two independent channels on PA4 (CH1) and PA5 (CH2). Both enabled simultaneously in the `dac_setup()` function.
- The project provides `stm32g051k6.ld` with MEMORY block: FLASH 32K at 0x08000000, RAM 18K at 0x20000000. Page 31 (last 1K of flash) is excluded from the FLASH region and reserved for NV parameter storage.

8.3 Docker toolchain

Build and flash are containerised. The Dockerfile installs `arm-none-eabi-gcc` and fetches `libopencm3` as a git submodule at build time. This ensures a reproducible build environment independent of the host OS.

- `docker compose run --rm brake build` — compiles firmware to `build/aart-brake.elf` and `build/aart-brake.bin`
- `docker compose run --rm brake flash` — flashes via OpenOCD over SWD (ST-Link V2 or compatible)
- `docker compose run --rm brake debug` — starts OpenOCD GDB server on port 3333

8.4 Build workflow

After cloning: `git submodule update --init --recursive` (fetches `libopencm3`), then `docker compose run --rm brake build`. The build produces `aart-brake.elf` (with debug symbols) and `aart-brake.bin` (raw binary for UART bootloader upload). Binary size is typically 8–12KB of the 32KB flash.

9. Pre-Commissioning Checklist

9.1 Hardware checks before first power-on

Work through these checks before applying track supply voltage for the first time:

- Confirm MCU is STM32G051K6U6 (QFN-32). PA4 and PA5 must route to GD2 and GD1 op-amp buffers respectively.
- Verify WHITE sense divider: R8=18k Ω , R9=4k7 Ω on PA0. At 16V gives 3.31V. If 15k Ω fitted instead of 18k, replace it — 15k gives 3.82V at 16V, exceeding ADC reference.
- Verify BLACK sense divider: R10=22k Ω , R11=4k7 Ω on PA1. Confirm tap is on MOTOR SIDE of D_PG. Essential for correct brake detection.
- Confirm D_PG (LTC4412) in BLACK controller path. Verify CTRL pin polarity from datasheet.
- Confirm D4 (MBRS360 or SS54) in series with Q1 output. D4 is essential whenever Mode A is used with a non-zero KA floor (brushless operation). Without D4, Q1 body diode takes repetitive ~46A current pulses when Q2 hard-brakes while Q1 is trying to maintain the floor.
- Confirm catch diodes D2/D3 are SS310 (100V). SS34 (40V) is not adequate on a 16V rail with inductive motor loads.
- Confirm F1 (polyfuse 500mA–1A) is in WHITE rail supply path only. Not in BLACK track path.
- Confirm J14 (voltmeter header) pin orientation: pin 1 = Vin (filtered PWM), pin 2 = GND, pin 3 = Vcc 3.3V. Confirm J_USB (USB-C) CC1 and CC2 pins each have 5.1k Ω pull-down to GND. Confirm D_USB (BAT54) is oriented with anode at VBUS/F_USB junction and cathode at LDO input. A reversed D_USB will feed track voltage directly to the USB host.

9.2 Scope verification sequence — Mode A (brake, all motors)

Step	Condition	Q1 — DAC2 (KA gate)	Q2 — DAC1 (brake gate)	Pass condition
1	Power on, V_BLACK = 8V (running)	3.3V (Q1 off)	0V (Q2 off)	Module transparent
2	V_BLACK drops to 0.3V (brake), pot CCW	3.3V	~1.5V DC (DAC_BRAKE_SOFT)	Soft brake. Smooth DC, no PWM.
3	Brake active, pot swept CW	3.3V	Ramps from ~1.5V to ~2.5V DC	Brake resistance decreasing
4	V_BLACK rises to 8V (throttle pickup)	3.3V	Ramps to 0V (trail release if set)	Release state then PASS

S t e p	Condition	Q1 — DAC2 (KA gate)	Q2 — DAC1 (brake gate)	Pass condition
5	Remove MCU power	3.3V	~1.8V (pull-up default)	Safe default brake. Car brakes.

9.3 Scope verification sequence — Mode B (brake + anti-brake, brushed)

S t e p	Condition	Q1 — DAC2 (KA gate)	Q2 — DAC1 (brake gate)	Pass condition
1	Running, V_BLACK = 8V	saved_ka_dac (floor) or 3.3V if not set	0V	Running. Q1 applies saved_ka_dac floor if set, else off.
2	V_BLACK drops to 0.3V (brake), pot CCW	3.3V (Q1 off during brake)	~1.5V DC	Soft brake. Q1 off during braking.
3	Brake active, pot CW	3.3V	~2.5V (hard brake)	Brake tracks pot. Q1 off.
4	Toggle to Mode C, pot CCW	0V (DAC_KA_OFF — no floor)	Linear per pot	KA tuning. Voltmeter shows 0V.
5	Toggle to Mode C, pot CW	~2V (DAC_KA_MAX)	Linear per pot	KA tuning. Voltmeter shows ~2V.
6	Toggle to Mode C, tune floor	ka_dac_table[pot_i dx] live	Normal linear brake	Pot dials KA floor. Voltmeter shows it.
7	Toggle back to Mode A (500ms → save)	saved_ka_dac applied as Q1 floor	Linear per pot	LED blinks 3×. Mode A racing with KA floor.

9.4 Scope verification sequence — Mode C (KA tuning, brushless)

S t e p	Condition	Q1 — DAC2 (KA gate)	Q2 — DAC1 (brake gate)	Pass condition
1	Toggle to Mode C	ka_dac_table[pot_i dx] live	Linear per pot (brake still works)	Voltmeter shows live KA floor voltage.
2	Pot CCW in Mode C	DAC_KA_OFF (0V — no floor)	Linear per pot	Voltmeter shows 0V. No floor.
3	Pot CW in Mode C	DAC_KA_MAX (~2V floor)	Linear per pot	Voltmeter shows ~2V. Q1 live for eCom verification.

S t e p	Condition	Q1 — DAC2 (KA gate)	Q2 — DAC1 (brake gate)	Pass condition
4	Toggle to Mode C, tune floor	ka_pending_save armed	500ms debounce	After 500ms: save + LED blinks 3×.
5	Mode A, V_BLACK = 8V	saved_ka_dac floor active	0V	Mode A running. KA floor applied by Q1.
6	V_BLACK drops (brake)	DAC_KA_OFF (Q1 off during braking)	Linear per pot	Braking. Q1 resumes floor after brake exit.

The most critical safety test in all modes is step 5 of the Mode A sequence: remove MCU power and verify Q2 enters its braking region via the pull-up circuit.

9.5 On-track tuning

Parameter	Command	Default	Tuning guidance
Minimum brake (pot CCW)	SET RAMP_MS <0 –200>	1860	0 = instant hard brake. 50–150ms for wing cars. 0–30ms for pan cars.
Anti-brake ceiling	Mode B pot position	DAC_KA_MAX	Coupled with brake via table. CCW = max anti-brake. CW = no anti-brake.
Keep-alive floor	Pot position when leaving Mode C	DAC_KA_OFF	Toggle to Mode C, rotate pot, toggle back to Mode A. Saves after 500ms.
KA floor save timing	SET BRAKE_ENTER <mV>	5000ms	Reduce if brake is slow to engage.
Brake entry threshold	SET BRAKE_ENTER <mV>	500mV	Reduce if brake is slow to engage.
Brake exit threshold	SET BRAKE_EXIT <mV>	1500mV	Raise if brake persists too long.
Brake exit threshold	Set minimum brake resistance in ohms; snaps to nearest stored table (2, 3, 4, 6 or 8Ω). Default 3Ω. Live immediately; SAVE to persist.	1500mV	Raise if brake persists too long.
Ramp-in time	SET RAMP_MS <0 –200>	0ms	0 = instant. 50–150ms for wing cars. 0–30ms for pan cars.
Trail release	SET RELEASE_MS_A/	0ms	0 = instant snap-off. 50–200ms for trail braking.

Parameter	Command	Default	Tuning guidance
	B/C <0–500>		
Latvian brake	SET LATVIAN_DVDT <mV/ms>	0 (off)	Enable with 1000. Watch telemetry for false triggers.

10. Three-Mode Design — Rationale and Behaviour

10.1 The braking problem

A slot car motor is a high-inductance load. When the throttle is cut, the motor acts as a generator and back-EMF drives current through the braid and track. An uncontrolled circuit allows the car to freewheel. A resistive brake shorts the motor through a resistor, converting kinetic energy to heat and providing a controllable deceleration force.

The module improves on a simple fixed resistor by providing: variable resistance (pot-controlled), a programmable minimum resistance floor, optional positive anti-brake for corner entry, and eCom keep-alive for brushless motors.

10.2 Mode A — Brake only, all motors

Pot maps linearly to Q2 brake: CCW = DAC_BRAKE_SOFT (~8Ω), CW = DAC_BRAKE_HARD (dead short). No lookup table. Q1 applies saved_ka_dac as a voltage floor whenever V_BLACK drops below it. Default saved_ka_dac = DAC_KA_OFF so Q1 stays off unless a floor has been set via Mode C. This makes Mode A suitable for both brushed motors (no floor) and brushless motors (with floor). Primary racing mode for all motor types.

10.3 Mode B — Brake and anti-brake (brushed)

Pot indexes brake_dac_table[] and ka_dac_table[] simultaneously. During braking, Q2 brakes at brake_dac_table[pot_idx] and Q1 stays off. After brake exit, Q1 injects anti-brake at ka_dac_table[pot_idx]. Both values are live from the same pot position via the coupled lookup table. CCW = minimum brake + maximum anti-brake (~2V). CW = maximum brake + no anti-brake. Brushed motors only.

No separate saved value. The anti-brake level is always the ka_dac_table value at the current pot index. The driver adjusts the pot to set the balance between brake and anti-brake. No capture gesture required.

10.4 Mode C — Keep-alive tuning (brushless, setup mode)

Mode C is a setup mode, not a racing mode. You enter it to set the KA floor voltage, then return to Mode A for racing. While in Mode C the voltmeter shows the live `ka_dac_table` value at the current pot position. Normal braking functions during Mode C (pot maps linearly to brake).

When Mode A is used with a non-zero `saved_ka_dac`, Q1 clamps `BLACK_to_track` above the floor whenever `V_BLACK` drops below it. The eCom sees a minimum voltage rather than a brownout. Mode C exists only to set this floor value conveniently while verifying the eCom response live.

On every toggle move to Mode C, the pot immediately dials the KA floor live (`ka_dac_table[pot_idx]`), both on Q1 and on the voltmeter. The driver rotates the pot until the eCom powers up reliably, then toggles back to Mode A. After 500ms the floor is saved. Normal braking via linear pot mapping continues during Mode C.

10.5 Keep-alive floor tuning procedure

- 1. Toggle to Mode C. The voltmeter shows the live `ka_dac_table` value at the current pot position.
- 2. Connect the car to the track with the PSU running. Rotate the pot and watch the voltmeter. CCW = no floor, CW = ~2V floor.
- 3. Rotate the pot CW slowly. Watch the voltmeter increase. Listen or watch for the eCom to power up (motor twitches, LED on eCom illuminates).
- 4. Continue CW until the eCom powers up reliably, then back off slightly CCW to find the minimum effective floor.
- 5. Toggle back to Mode A. After 500ms the floor is saved. LED blinks 3×. Voltmeter returns to track voltage.
- 6. Verify Mode A braking with KA floor active. The car is ready for brushless racing.

11. FET Behaviour Reference

11.0 Linear mode operation — why and how

Both Q1 and Q2 are operated in their linear (triode) region rather than as switches. This is the most important aspect of the output stage design and deserves a full explanation.

11.0.1 The three regions of a MOSFET

Region	Condition	Behaviour	Used for
Cut-off	$V_{gs} < V_{th}$	No drain current. FET is an open circuit.	FET off — passthrough mode
Saturation	$V_{gs} > V_{th}$, $V_{ds} > V_{gs} - V_{th}$	Drain current controlled by V_{gs} only. FET acts as a current source.	Switching (digital)
Linear (triode)	$V_{gs} > V_{th}$, $V_{ds} < V_{gs} - V_{th}$	Drain current controlled by both V_{gs} and V_{ds} . FET acts as a voltage-controlled resistor.	Variable resistance — this design

In the linear region: $R_{ds(on)} \propto 1 / (V_{gs} - V_{th})^2$. For IRLB3034 (Q2), $V_{th} \approx 1.5V$. The useful control window is $V_{gs} \approx 1.5V$ to $2.5V$ (600mV). The 12-bit DAC gives 750 steps over this window at 0.8mV per step — finer resolution than any mechanical pot.

11.0.2 Why linear mode, not PWM switching

- Brushless eCom compatibility. 20kHz switching on the braid creates ripple that confuses the eCom ADC and can cause misfires or resets. Linear FET presents smooth, continuous resistance.
- EMC regulations. High-frequency switching requires CISPR 32 / EN 55032 compliance. A slot car track is a long antenna. Linear control has no switching frequency.
- Resolution. PWM effective resistance depends on motor inductance ($L/\text{duty} \times T$), which varies with motor and speed. DAC-driven linear control is independent of motor parameters.

11.0.3 The gate drive challenge in linear mode

- Stability. FET in linear mode has negative thermal coefficient — self-stabilising. Op-amp buffer must be stable with capacitive gate load. Gate resistors R3/R4 (10Ω) isolate op-amp from gate capacitance.
- Safe default. Pull-up must land in the linear region (V_{gs} between V_{th} and full enhancement), not just force gate high. For Q2 (IRLB3034, $V_{th} \approx 1.5\text{--}2V$), pull-up target is $\sim 1.8\text{--}2.0V$ — light-to-medium braking, not violent dead-short.

The safe-default pull-up for Q2 intentionally targets the lower end of the linear region. The car brakes but not violently. For competition use, a harder default can be set by adjusting the pull-up divider.

11.1 Q1 — Anti-brake / keep-alive FET (P-channel, high-side)

Q1 (IRF9Z34N, P-channel) connects WHITE rail to BLACK_to_track. Source at WHITE rail (12–16V). Gate driven by DAC2 via op-amp buffer GD1.

DAC2 output	Vgate	Vgs (source at 12V)	Q1 state	Effect
4095 (3.3V)	3.3V	$3.3 - 12 = -8.7V$	Fully ON (well past V_{th})	WARNING: see open items. Gate must reach ~12V to turn Q1 off.
2480 (~2.0V)	~2.0V	–9.7V	Maximum conduction	2V anti-brake injection or KA floor.
3720 (~3.0V)	~3.0V	–8.7V to –9V	Light conduction	Light anti-brake or soft KA floor.

Q1 turn-off resolved by the GD1 floating supply (section 7A.7). GD1 (LM321) is referenced to the WHITE rail via ZD1 (15V zener), R_Z1 and C_Z1, so its output swings from WHITE–15V (Q1 fully on) to WHITE (Q1 off, $V_{gs} \approx 0V$). DAC_KA_OFF = 4095 now produces correct turn-off. Bench-verify the turn-off margin at 12V rail before PCB commit.

11.2 Q2 — Brake FET (N-channel, low-side)

Q2 (IRLB3034, N-channel) connects BLACK_to_track to RED/GND. Gate driven by DAC1 via op-amp buffer GD2.

DAC1 output		Vgate	Q2 state	Rds approx	Motor sees
0 (DAC_BRAKE_OFF)		0V	Off ($V_{gs} < V_{th}$)	Open circuit	No brake
1860 (DAC_BRAKE_SOFT)		~1.5V	Linear region onset	Several Ω	Soft brake (selectable 2/3/4/6/8 Ω)
Intermediate		~2.0V	Mid linear	~0.5 Ω	Medium brake
3095 (DAC_BRAKE_HARD)		~2.5V	Deep linear / saturation boundary	~5m Ω	Near dead short
PA0	ADC_IN0	WHITE sense		R8/R9 divider. 3.31V max at 16V rail.	
PA1	ADC_IN1	BLACK_to_track sense		R10/R11 divider. Motor side of D_PG.	
PA3	ADC_IN3	Pot wiper		10k linear pot, 0–3.3V. No centre detent.	
PA4	DAC_CH1	Q2 brake gate → GD2		0–3.3V. DAC_BRAKE_OFF to DAC_BRAKE_HARD.	

DAC1 output		Vgate	Q2 state	Rds approx	Motor sees
PA5	DAC_CH2		Q1 KA gate → GD1	0–3.3V. DAC_KA_MAX to DAC_KA_OFF.	
PA9	USART1_TX AF1		UART TX	J11. 115200 8N1.	
PA10	USART1_RX AF1		UART RX	J11.	
PA13	SWDIO AF0		SWD debug	J10.	
PA14	SWDCLK AF0		SWD clock	J10.	
PB0	GPIO IN		Toggle bit 0	Active LOW, internal pull-up.	
PB1	GPIO IN		Toggle bit 1	Active LOW, internal pull-up.	
PB3	GPIO IN		(reserved)	Previously capture button. Unused in Rev 11. Reserved for future use.	
PB4	TIM3_CH1 AF1		Voltmeter PWM	1kHz. RC filter 10k/100µF → J14 pin 1.	
PB6	GPIO OUT		Status LED	Blinks 3× on flash save.	
PB8	BOOT0		ROM bootloader	J12. Hold + NRST for field update.	

12.3 Toggle switch decoding

PB0 (bit 0)	PB1 (bit 1)	Mode	Behaviour
HIGH (released)	HIGH (released)	Mode A	Brake only. Pot → brake_dac_table[]. Q1 off.
LOW (pressed)	HIGH (released)	Mode B	Brake + anti-brake. Pot → brake_dac_table[] and ka_dac_table[] simultaneously.
HIGH (released)	LOW (pressed)	Mode C	Brake + KA floor. On toggle entry, pot index read once → saved_ka_dac.
LOW (pressed)	LOW (pressed)	Invalid	Defaults to Mode A (safe).

13.1 Industry context

In slot car racing, ‘brake’ refers to the resistance applied across the motor when the trigger is released. ‘Ramp-in’ refers to how quickly that brake resistance builds. Hard brake (instant full resistance) gives maximum deceleration but can cause the car to snap sideways if the driver is not precise. Soft brake (gradual ramp) is more forgiving and faster for most drivers on most tracks.

13.2 Why positive anti-brake helps

After the brakes release and the driver begins to pick up throttle, there is a brief period where the car's forward momentum has not yet been converted to acceleration. A small positive voltage injection from Q1 provides a forward torque impulse during this transition, helping the car stay planted and balanced on corner exit. Commercial controllers describe this as anti-brake or positive anti-brake, typically in the range 0.2–2V.

13.3 Brake ramp-in

13.3.1 Exponential ramp shape

The ramp-in is exponential rather than linear. Each 1ms tick: $\text{brake_dac} += (\text{target} - \text{current}) \times \alpha / 256$. Alpha = 256 for instant (no ramp), $\alpha \approx 3$ for maximum ramp (200ms). This gives fast initial bite followed by a gentle tail, which matches what drivers feel as natural.

13.3.2 Setting the ramp time (UART only)

Ramp time is a UART-only parameter. The pot is not used to set ramp time — it controls brake and anti-brake levels directly. Use UART:

- SET RAMP_MS <0–200> — 0 = instant hard brake (default). 50–150ms suits most wing cars. 0–30ms suits pan cars.
- SAVE — persists to flash.
- GET — shows current value.

13.3.3 Interaction with positive anti-brake

The ramp-in applies on brake entry. Anti-brake (Mode B) applies on brake exit. They operate at different times and do not interact. A long ramp-in (gradual brake buildup) combined with a moderate anti-brake (gentle exit impulse) gives the most forgiving driving characteristic for club-level drivers on brushed motors.

13.4 Flash storage layout

Double-word	Offset	Contents
DW0	0x00	magic (32-bit) ka_dac (16-bit) ramp_ms (16-bit)
DW1	0x08	0xFFFF brake_enter_mv (16-bit) brake_exit_mv (16-bit)
DW2	0x10	0xFFFF brake_ohms_idx (16-bit) latvian_dvdt (16-bit)
DW3	0x18	rel_ms_c (16-bit) rel_ms_b (16-bit) 0xFFFF rel_ms_a (16-bit)

Magic = 0xAA270008 (Rev 11). Four double-words (DW0–DW3) store ka_dac, ramp_ms, brake thresholds, brake_ohms_idx, latvian_dvdt, and the three release times. Old records with a different magic are silently discarded and firmware defaults apply until SAVE is issued.

13B. Trail Brake Release

13B.1 Concept

Trail braking is the technique of maintaining partial brake pressure while beginning to apply throttle at corner entry. The brakes fade out smoothly as the car transitions from braking to acceleration. The module implements this as a configurable linear release ramp on brake exit.

13B.2 Entry vs release — different shapes by design

Brake entry is exponential (fast initial bite, decaying tail). Brake release is linear (constant fade rate). The linear release is deliberate — a predictable, constant fade that a driver can feel and modulate. An exponential release would feel unpredictable.

Phase	Shape	Rate	Why
Brake entry (ramp-in)	Exponential	Fast initial onset, decaying tail	Immediate grip; gentle progression prevents snap.
Brake release (trail)	Linear	Constant rate per millisecond	Predictable fade the driver can trust during corner entry.

13B.3 Configuration

Command	Default	Range	Typical starting points
SET RELEASE_MS_A <ms>	0 (instant)	0–500ms	Pan cars: 50–100ms. Wing cars: 100–200ms.
SET RELEASE_MS_B <ms>	0 (instant)	0–500ms	Same guidance as mode A.
SET RELEASE_MS_C <ms>	0 (instant)	0–500ms	Brushless: 30–80ms. Longer values may cause eCom issues.

13B.4 Full throttle snap override

If `V_BLACK` rises above `BRAKE_EXIT_MV + RELEASE_SNAP_MV` (500mV) — indicating hard throttle application — the release ramp is overridden and the brake snaps immediately to off. Trail braking only operates in the light-throttle transition zone.

13B.5 Telemetry

State field shows `RELEASE` while trail ramp is active, `PASS` when complete. Monitor via UART telemetry to tune release time.

14B. Latvian Brake — Emergency Stop on Track Call

14B.1 Concept and use case

The primary use case is a car coming out of the slot — loss of braid contact. The eCom or ESC loses its voltage reference and behaves erratically. A secondary use case is rapid braking on a track call (PSU cut). The firmware detects either event via `WHITE` rail dV/dt monitoring.

14B.2 Trigger — `WHITE` rail dV/dt

The `WHITE` rail voltage is sampled each 1ms tick. If the voltage drops faster than the `latvian_dvdt_mv_per_ms` threshold (in mV per ms), the Latvian brake fires immediately. This detects PSU cut (rapid rail collapse) and car-out-of-slot (controller no longer loaded, rail rises briefly then collapses).

14B.3 Behaviour when active

On Latvian brake trigger: `DAC_BRAKE_HARD` applied immediately. Q1 turned off. State = `STATE_LATVIAN_BRAKE`. All normal mode logic, ramp-in, anti-brake, and KA floor are bypassed. The hardware pull-up on GD2 holds Q2 in braking region even after MCU browns out.

14B.4 Configuration via UART

- `SET LATVIAN_DVDT <mV/ms>` — enable with a value such as 1000. 0 = disabled (default).
- Too sensitive: false triggers during normal braking. Increase threshold.
- Too insensitive: misses out-of-slot events. Decrease threshold.
- Recommended starting point: `SET LATVIAN_DVDT 1000`, then observe telemetry during normal braking to verify no false triggers.

14B.5 Telemetry

State field shows LATVIAN while active. Module exits LATVIAN state when V_WHITE recovers above RAIL_UNDERVOLTAGE_MV (5000mV).

14. UART Debug Interface and Field Flashing

14.1 Overview

USART1 on PA9 (TX) and PA10 (RX) provides a 115200 8N1 serial interface via J11. Two functions: CSV telemetry output at ~10Hz, and a runtime command interface. No bootloader required for parameter changes — all SET commands take effect immediately and SAVE persists to flash.

14.2 SWD debug header (J10)

4-pin 2.54mm header: pin 1 = SWDIO, pin 2 = SWDCLK, pin 3 = GND, pin 4 = +3.3V. Compatible with ST-Link V2, CMSIS-DAP, and J-Link. Used for initial flashing and GDB debugging.

14.3 UART telemetry header (J11)

3-pin 2.54mm header: pin 1 = TX (PA9), pin 2 = RX (PA10), pin 3 = GND. Connect to any USB-UART adapter. Telemetry format: CSV with header line beginning #. State names: PASS, RAMP, BRAKE, RELEASE, AB (anti-brake), KA (keep-alive only), KA_TUN (Mode C tuning), LATVIAN, SAFE.

14.4 Runtime commands

Command	Example	Description
GET ID	GET ID	Returns firmware identity string: #ID: AART-BRAKE-v11-STM32G051
GET	GET	Returns all current parameter values.
SET BRAKE_ENTER <mV>	SET BRAKE_ENTER 500	Brake entry threshold (V_BLACK falling below this = brake active).
SET BRAKE_EXIT <mV>	SET BRAKE_EXIT 1500	Brake exit threshold (V_BLACK rising above this = brake released).

Command	Example	Description
SET KA_MAX <dac>	SET KA_MAX 2480	Maximum KA/anti-brake conduction DAC value.
SET LATVIAN_DVDT <mV/ms>	SET LATVIAN_DVDT 1000	Latvian brake dV/dt threshold. 0 = disabled.
SET RAMP_MS <ms>	SET RAMP_MS 80	Brake ramp-in time 0–200ms. UART-only — not settable by pot.
SET RELEASE_MS_A <ms>	SET RELEASE_MS_A 150	Trail release time for Mode A (0–500ms).
SET RELEASE_MS_B <ms>	SET RELEASE_MS_B 100	Trail release time for Mode B (0–500ms).
SET RELEASE_MS_C <ms>	SET RELEASE_MS_C 50	Trail release time for Mode C (0–500ms).
SAVE	SAVE	Saves all current parameters to flash. LED blinks 3×.
HELP	HELP	Prints command list.

14.5 BOOT0 and NRST buttons

J12 (BOOT0, PB8) and J13 (NRST) are 2-pin 2.54mm headers with momentary pushbuttons. To enter the STM32 ROM UART bootloader: hold BOOT0, press and release NRST, release BOOT0. The device will enumerate on USART1 at 115200 8N1 and accept standard STM32 bootloader protocol frames.

14.6 ROM UART bootloader procedure

- 1. Connect a USB-C cable to the host PC (preferred — uses the onboard CH340N bridge), or a USB-UART adapter to the J11 header (TX, RX, GND) as an alternative. See section 7A.9.6 for the USB-C path.
- 2. Hold the BOOT0 button (J12). Press and release NRST (J13). Release BOOT0.
- 3. Use stm32flash or STM32CubeProgrammer to upload aart-brake.bin.
- 4. Press NRST to restart the device. The firmware runs immediately.

15. Potentiometer Resolution and Control Precision

15.1 The signal chain

Pot wiper (0–3.3V) → PA3 ADC (12-bit, 4096 steps) → firmware maps ADC value to DAC value (12-bit, 4096 steps) → DAC output → op-amp buffer → FET gate voltage → $R_{ds(on)}$ → effective brake resistance.

15.2 Effective resistance — how linear braking works

The FET $R_{ds(on)}$ varies as approximately $1/(V_{gs}-V_{th})^2$ in the linear region. The useful Q2 control window is ~600mV (1.5V to 2.5V gate). This maps to a wide range of resistance from ~8 Ω (soft) to ~5m Ω (dead short). The relationship is highly non-linear — most of the useful variation is in the lower half of the DAC range.

15.3 Resistance resolution across the brake range

At mid-range (DAC $\approx$ 2400): $dR_{ds}/dDAC \approx 0.05\Omega/\text{step}$. At soft end (DAC $\approx$ 1900): $dR_{ds}/dDAC \approx 0.2\Omega/\text{step}$. At hard end (DAC $\approx$ 3000): $dR_{ds}/dDAC \approx 0.001\Omega/\text{step}$. The pot full range (4096 ADC steps) maps to the full 750 DAC steps in the control window — approximately 5:1 mechanical advantage, giving finer effective resolution than the pot travel alone.

15.4 Anti-brake voltage resolution

The Q1 anti-brake control window is also ~600mV (2.0V to 3.3V gate, inverted). 750 DAC steps across this window. Pot full travel CCW to CW maps 0V to ~2V of anti-brake injection voltage. Resolution: 2V / 750 steps $\approx$ 2.7mV per pot step — well below any perceptible threshold.

15.5 The binding constraint: DAC quantisation

With a 12-bit DAC and 750 useful steps in the brake control window, the limiting factor is DAC quantisation (0.8mV/step), not pot mechanical resolution. A standard 10k Ω linear pot has typically 1–10 Ω end-to-end resistance variation and $\pm 1\%$ linearity — far coarser than 0.8mV DAC steps. No centre detent is needed or useful.

15.6 Pot specification recommendations

- Linear taper (not audio/log). The linear taper gives uniform resistance change per degree of rotation.
- 10k Ω end-to-end resistance. Matches ADC input impedance; minimal loading on 3.3V reference.

- No centre detent. A plain linear 10kΩ pot. CCW = minimum brake (and maximum anti-brake in Mode B). CW = maximum brake (no anti-brake). No mode boundary at centre.
- Dust-sealed (IP54 or better) recommended for module enclosure use. The module will be handled at the track.
- Shaft diameter and knob style: to taste. Standard 6mm D-shaft fits most available knobs.

15B. Comparison with Richard Lang's Approach and Table Resolution

15B.1 Richard Lang's implementation

The AART Rev 11 lookup table approach (used in Mode B) is inspired by Richard Lang's earlier Arduino-based slot car controller implementation. Understanding the similarities and differences clarifies the design decisions made in the AART module.

Richard's implementation (210922PeteRCarrayAntiBrake) uses a 256-entry PROGMEM array `adcCurve[]` stored in AVR flash memory to give a ratiometric (exponential) characteristic to the brake, acceleration, and deceleration pots. The 8-bit ADC reading is used directly as an array index:

- `brake output = adcCurve[brakeADC]` — maps 0–255 pot reading to 16–255 output with ~16:1 exponential ratio
- anti-brake is handled by a second, separate pot (`antibrkADC`, the repurposed decel pot) producing an independent floor voltage
- the two outputs are combined in `minmaxfunc()`: when the trigger is in the brake zone, instead of returning 0V, the function returns `antibrklimit = min(minimumlimit, antibrkADC)`

Aspect	Richard Lang (Arduino)	AART Rev 11 (STM32G051)
Microcontroller	ATmega32U4 (8-bit AVR, 8MHz RC osc)	STM32G051K6U6 (32-bit Cortex-M0+, up to 64MHz)
Table storage	PROGMEM flash (<code>pgm_read_byte_near()</code>)	RAM or flash constant array, no special access required
Table entries	256 × 1 byte = 256 bytes per table	256 × 2 bytes = 512 bytes per table (<code>uint16_t</code> for 12-bit DAC)
ADC resolution	8-bit (256 steps)	12-bit ADC, right-shifted to 8-bit index (256 steps)
Brake output	8-bit PWM (0–255)	12-bit DAC (0–4095, effective range 1860–3095)
Anti-brake input	Separate pot (<code>antibrkADC</code>)	Same pot as brake — inverse relationship in table

Aspect	Richard Lang (Arduino)	AART Rev 11 (STM32G051)
Anti-brake output	PWM floor set by antibrklimit	DAC floor set by ka_dac_table[idx]
Pot count	2 pots (brake + anti-brake)	1 pot (both outputs from single index)
Capture gesture	None — anti-brake follows pot live	None — both outputs follow pot live (KA floor captured by toggle)
Table shape	Exponential ratio ~16:1 (values 16–255)	Exponential (gamma=1.8), brake 1860–3095, KA 2480–4095

15B.2 Key design differences

The most significant difference is the relationship between brake and anti-brake:

- Richard’s design: brake and anti-brake are independent. The driver can set hard braking and high anti-brake simultaneously if desired. The two pots allow the characteristics to be tuned independently.
- AART Rev 11: Mode B couples brake and anti-brake through the table (same pot, both outputs live). Mode A is linear brake only for all motors, with saved KA floor from Mode C applied by Q1. Mode C is KA tuning only.

This coupling is deliberate. In slot car racing, very hard braking and very high anti-brake are physically contradictory objectives — the anti-brake torque impulse would immediately counteract the deceleration the driver is trying to achieve. Coupling them through the table enforces a physically sensible relationship with a single control.

The tradeoff is reduced flexibility. A driver who wants to experiment with unconventional combinations (e.g. moderate brake with high anti-brake) cannot do so with a single pot. The UART interface provides RAMP_MS and RELEASE_MS as compensation — the parameters most likely to need individual tuning are accessible without a physical pot dedicated to each.

15B.3 Should the table be expanded beyond 256 entries?

The STM32G051 has a 12-bit ADC (4096 steps) and 32KB flash. The question of whether to use all 4096 ADC steps as table indices rather than right-shifting to 8 bits (256 steps) deserves a rigorous answer.

15B.3.1 What limits effective control resolution?

Source of variation	Approximate magnitude	Equivalent ADC steps (12-bit)	Implication
Pot mechanical resolution (human hand, $\pm 1^\circ$ of 270° travel)	$\pm 0.37\%$ of range	± 15 steps	Hand cannot reliably set finer than ~ 15 ADC steps
Pot tracking resistance tolerance ($\pm 1\%$ typical)	± 41 steps	± 41 steps	Pot linearity limits meaningful resolution to ~ 100 steps
FET $R_{ds(on)}$ thermal variation ($\pm 20\%$ with temperature)	± 247 DAC-equivalent steps	$\gg 100$ steps	Gate voltage must change significantly before R_{ds} changes noticeably
ADC noise + quantisation	$\pm 2-3$ LSB	± 3 steps	Negligible — not the limiting factor
Table step size at soft end (256 entries)	~ 1 DAC count per step	1 step	Already at the floor of perceptibility

The analysis shows that the human hand and pot mechanics limit effective resolution to approximately ± 15 ADC steps at best. With 256 table entries spanning the full 12-bit ADC range, each table step corresponds to 16 ADC counts — comfortably above the noise floor but below the hand/pot resolution limit. This is the correct operating point.

15B.3.2 Cost of a 4096-entry table

If the ADC were used at full 12-bit resolution with a 4096-entry table:

- Table size: 4096×2 bytes (uint16_t) $\times 2$ tables (brake + KA) = 16,384 bytes = 16KB. Exactly half the STM32G051's 32KB flash budget.
- Step size at soft end: ~ 0.07 DAC counts per ADC step — sub-quantisation, producing long runs of identical table values. Entirely imperceptible.
- Build time: the table would need to be pre-computed and stored as a 16KB constant, or computed at runtime (feasible on a 32-bit MCU, ~ 4 ms at 64MHz for two pow() loops).

A 4096-entry table costs 16KB of flash for zero perceptible improvement. The 256-entry table is the correct choice. The 32-bit MCU's real advantage over the 8-bit AVR is not wider tables — it is the 12-bit DAC (vs 8-bit PWM), the ability to compute table values at runtime rather than storing them in PROGMEM, and the availability of sufficient GPIO for the full circuit without pin-sharing compromises.

15B.3.3 What the 32-bit MCU actually enables

The architectural advantages of the STM32G051 over an 8-bit AVR for this application are:

- 12-bit DAC output. Richard's design uses 8-bit PWM (256 levels) filtered to analogue. The G051 generates a true DC voltage from the DAC with 4096 levels, no filtering capacitor, no PWM ripple on the motor terminal.
- Two independent DAC channels on dedicated pins (PA4 and PA5). No pin-sharing, no timer multiplexing.
- Linear mode gate drive. The DAC outputs a stable DC voltage to the op-amp gate driver. An 8-bit MCU driving a gate driver from PWM requires a filter capacitor and creates residual ripple at the switching frequency.
- Runtime table computation. On the G051, `brake_dac_table[]` and `ka_dac_table[]` can be computed in `setup()` using `powf()` rather than stored as PROGMEM constants. This costs ~2ms at startup and saves 1KB of flash. (Currently stored as constants for simplicity.)
- Sufficient GPIO without repurposing ADC pins. Richard's design repurposed the decel pot ADC channel for anti-brake because the AVR ran out of pins. The G051 QFN-32 has enough GPIO for dedicated toggle and LED pins without ADC conflicts.

7C. MCU Selection Review — Could We Use a Smaller Device?

7C.1 Requirements recap

Before evaluating alternatives it is useful to state the minimum peripheral requirements for this application:

Peripheral	Minimum requirement	Why
DAC	2 channels, 12-bit resolution	Gate voltage for Q1 and Q2 in linear mode. 8-bit DAC gives only ~46 steps in the 600mV control window — noticeable coarseness in the soft brake region.
ADC	3 channels, 12-bit	WHITE rail sense (PA0), BLACK_to_track sense (PA1), pot wiper (PA3).
GPIO in	3 pins minimum	PB0/PB1 toggle, PB3 reserved.
GPIO out	2 pins	PB4 voltmeter PWM (TIM3_CH1), PB6 LED.
Timer	1 channel PWM	TIM3_CH1 for voltmeter RC-filtered PWM output.
UART	1x USART	PA9/PA10 for debug telemetry and field parameter updates.
SWD	2 pins	PA13/PA14 for programming and debug.
Flash	~12KB firmware + 1KB NV	Rev 11 firmware is approximately 10–12KB compiled.
RAM	~2KB	Control loop, UART buffers, context structure.

The critical constraint is the dual 12-bit DAC. This is not present on all members of the STM32G0 family and is the primary driver of the current MCU choice.

7C.2 Could we use a smaller STM32G0 package?

The STM32G051 is available in five package sizes: TSSOP20 (F suffix), UFQFPN28 (G suffix), LQFP32 / UFQFPN32 (K suffix), LQFP48, and UFQFPN48. The current device is the K6U6 — UFQFPN32 (5×5mm QFN-32). The F6P6 variant in TSSOP20 (6.5×4.4mm) is the same silicon and appears smaller. However:

The TSSOP20 package has a GPIO bonding problem. In that package, multiple port pins share a single physical pad because there are not enough physical pins to expose them all independently. Specifically: PB3, PB4, PB5, and PB6 are bonded to the same pad, and PB0, PB1, PB2, and PA8 share another pad.

This design uses PB0 and PB1 (toggle switch bits), PB4 (voltmeter PWM output), and PB6 (status LED) — all four from the bonded group. They cannot all be used simultaneously from a single physical pin. The TSSOP20 is therefore not a viable package for this circuit without a complete pin reassignment that may not be achievable within the 20-pin constraints.

The UFQFPN32 (QFN-32) used in the current design is 5×5mm with 0.5mm pitch. This is actually easier to solder with hot air than TSSOP20 (0.65mm pitch, 20 pins with exposed leads) because the thermal pad anchors the part and the QFN pads are more tolerant of slight misalignment. The package size difference is marginal and the TSSOP20 offers no assembly advantage for this application.

7C.3 Could we use a DAC-less MCU with an external DAC?

The DAC-less members of the STM32G0 family (G030, G031, G041) are available in smaller and cheaper packages including TSSOP20 and SO-8N (8-pin SOIC). Using one of these with an external dual DAC IC such as the MCP4822 (SPI, 12-bit, dual channel) would in principle work. The trade-offs:

Aspect	STM32G051K6U6 (current)	G031 + external MCP4822
MCU cost	~€0.80 (JLCPCB)	~€0.50 (G031 TSSOP20)
DAC cost	€0 (integrated)	~€0.60 (MCP4822 SOT-23-8)
Total BOM delta	—	+€0.30 — no saving
Extra components	None	MCP4822 + 3× decoupling caps + SPI wiring
Extra GPIO consumed	None	3 (SCK, MOSI, CS) — reduces available GPIO
SPI bus contention	None	Must not conflict with any other SPI device

Aspect	STM32G051K6U6 (current)	G031 + external MCP4822
Startup behaviour	DAC outputs valid immediately at power-on	SPI init required before DAC is valid — gate driver pulls up before MCU sets output
Firmware complexity	dac_setup() sets PA4/PA5 directly	SPI driver + MCP4822 register protocol required
Safe default	Pull-up on op-amp input handles power-off	Same, but startup race condition more complex
libopencm3 support	Full G051 DAC support in tree	SPI library available but external DAC needs manual driver

The conclusion is clear: the external DAC approach costs more (not less), uses more GPIO, adds firmware complexity, creates a startup race condition with the gate driver pull-up, and provides no benefit. It exists as a theoretical option only.

7C.4 Is the 32-bit CPU necessary?

The CPU word size (32-bit vs 8-bit) is unrelated to DAC resolution. The DAC peripheral in the G051 is 12-bit regardless of the CPU. The question of whether an 8-bit MCU with an integrated 12-bit DAC could be used is largely academic — no widely available, JLCPCB-stocked 8-bit MCU has a dual 12-bit DAC integrated. The 8-bit AVR family (used in Richard Lang's reference implementation) has no integrated DAC at all; it uses PWM filtered to analogue, which is specifically what the AART design avoids due to switching noise on the eCom braid.

The genuine advantages of the STM32G051 over an 8-bit AVR for this application are:

- Integrated 12-bit dual DAC. True DC voltage output, no filter capacitor, no PWM ripple on the motor terminal. This is the single most important capability difference.
- Two independent DAC channels on dedicated pins (PA4, PA5). No pin-sharing, no timer multiplexing, no interaction between Q1 and Q2 gate drive.
- Stable DC gate drive. The op-amp buffer receives a steady DC input. No residual switching frequency at the gate, no interference with brushless eCom electronics.
- Sufficient GPIO without pin-sharing compromises. Richard Lang's Arduino implementation had to repurpose the decel pot ADC channel for anti-brake because the ATmega32U4 ran out of dedicated pins. The G051 QFN-32 has enough GPIO for all signals with pins to spare.
- ROM UART bootloader. Field firmware updates without ST-Link — just BOOT0 + NRST and a USB-UART adapter. The ATmega32U4 required avrdude and a specific COM port setup.
- libopencm3 toolchain. Deterministic, no RTOS, no hidden ISR overhead, Docker build, reproducible environment.

7C.5 Recommendation

The STM32G051K6U6 QFN-32 is the correct MCU for this application. It is the smallest package of the only STM32G0 variant that has an integrated dual DAC, sufficient GPIO for all required signals without pin bonding constraints, and is available at JLCPCB. No alternative MCU offers a meaningful advantage:

- Smaller STM32G051 packages (TSSOP20) cannot expose the required GPIO pins simultaneously due to bonding.
- DAC-less MCUs with external DAC are more expensive, more complex, and no smaller in overall board area.
- 8-bit MCUs with integrated DAC are not available in this combination at JLCPCB at competitive price points.

The G051K6U6 at ~€0.80 in QFN-32 represents the optimal point in the cost/capability/size trade-off space for this circuit. The 32-bit CPU, 32KB flash, and 18KB RAM are all substantially in excess of what the firmware requires — but the peripheral set (dual 12-bit DAC + ADC + UART + SWD + timer) is precisely what is needed, and there is no smaller or cheaper part that provides all of it in a single package.

Rev	Date	Author	Changes
25.0	Jun 2026	AART	Rev 11 firmware + document. Mode A: brake only all motors (linear pot, saved_ka_dac KA floor applied by Q1 — makes Mode A safe for brushless when floor set). Mode B: brushed anti-brake, lookup table couples brake + anti-brake simultaneously. Mode C: KA tuning only (brushless), pot dials floor live on Q1 and voltmeter, save on leaving after 500ms debounce. D4 essential for brushless Mode A. All stale mode descriptions, SET button references, handback timer references, and capture state references removed from sections 2, 3.6, 9, 10, 12, 13, 14. Figure 2 SVG redrawn for Rev 11.
24.2	Jun 2026	AART	Added section 7C: MCU selection review. Covers peripheral requirements, STM32G051 TSSOP20 GPIO bonding problem (PB3-PB6 bonded, rules out TSSOP20), external DAC option analysis (no cost saving, more complex, startup race), 32-bit vs 8-bit CPU discussion, and recommendation confirming G051K6U6 QFN-32 as optimal choice.
24.1	Jun 2026	AART	Added section 15B: comparison with Richard Lang Arduino implementation (table, two-pot vs single-pot, coupled vs independent outputs), and table resolution analysis (why 256 entries is correct on a 32-bit MCU, 4096-entry table costs 16KB for zero improvement, what the G051 actually enables over AVR).
24.0	Jun 2026	AART	Rev 10 firmware. Radical simplification: single pot indexes into 256-entry lookup tables (brake_dac_table[], ka_dac_table[]) producing both brake and anti-brake/KA outputs simultaneously. No capture button, no handback timer, no frozen brake, no capture states. KA floor captured by toggle move to MODE_KA (reads pot index once, saves, LED blinks, 2-sec voltmeter display). SET button (PB3) removed. saved_antibrake_dac, saved_min_brake_dac, saved_handback_ms, frozen_brake_dac, capture fields all removed. Flash magic 0xAA270008. Sections 2, 3.6, 4, 9, 10, 12, 13, 14, 15 updated. Lookup

Rev	Date	Author	Changes
			table design inspired by a previous implementation by Richard Lang.
22.0	Jun 2026	AART	Major UX redesign: pot controls voltage setpoints in all modes. Mode A pot = brake resistance during braking, capture button + pot = anti-brake ceiling. Mode B pot = brake floor, button captures it. Mode C unchanged. Ramp-in and trail release UART-only. Firmware: handle_capture() split into per-mode handle_capture_a/b/c(). BrakeCtx_t adds saved_antibrake_dac, saved_brake_floor_dac, capture preview fields. Flash magic 0xAA270005, DW4 added. SVG diagram AART_Brake_UI.svg added.
21.0	Jun 2026	AART	Added mini voltmeter interface: section 3.7 with display mode table, circuit description (TIM3_CH1 PB4, R_VM 10k, C_VM 100uF, D_V SS310), J14 3-pin connector pinout, firmware PWM scaling. BOM updated. KiCad updated.
20.1	Jun 2026	AART	Added section 7A.6: Power-down, brownout and hold-up capacitor. STM32G051 POR/PDR/BOR supervisor table. Current budget comparison. Worked hold-up cap calculation ($C = I \times t / \Delta V$, 100µF). Component selection table. Power-down sequence table.
20.0	Jun 2026	AART	Section 11 completely rewritten. New section 11.0: linear mode operation — three MOSFET regions, why linear not PWM, gate drive stability and safe-default challenge. Sections 11.1/11.2/11.3 rewritten with correct DAC values and D4 current path analysis.
19.2	Jun 2026	AART	GD1 ordered before GD2 in component table.
19.1	Jun 2026	AART	Q1 ordered before Q2 in all tables. Consistency audit fixes: section 7B CCR→DAC language, capture button PA5→PB3, flash record description, safe-default mechanism; section 8.2 linker script stm32g041j6→stm32g051k6, RAM 8K→18K; section 9 DAC column headers and data rows reordered.
19.0	Jun 2026	AART	Added section 3.6 User Interface. Removed section 7.0 prototype strategy table. Section 9 completely rewritten for DAC-based verification.
18.0	Jun 2026	AART	Added trail brake release (section 13B): linear release ramp on brake exit, configurable per mode via SET RELEASE_MS_A/B/C (0-500ms). Added GET ID UART command. Flash magic 0xAA270004. STATE_RELEASE added.
17.0	Jun 2026	AART	Added section 1.3 Circuit Topology with Figure 1 (AART_Brake_Topology.svg). Six explanatory bullets. Topology diagram matches hand-drawn schematic.
16.0	Jun 2026	AART	Production design adopted. TIM3 PWM removed, replaced by DAC1 CH1/CH2. D_PG LTC4412 in BLACK path. R10/R11 BLACK sense on motor side of D_PG. Toggle/capture moved to PB0/PB1/PB3. Firmware Rev 7.
15.3	Jun 2026	AART	Section 1.1 Purpose rewritten: three design drivers (fine control, Group 12 anti-brake, eCom keep-alive floor).
15.2	Jun 2026	AART	Corrected conflation of positive anti-brake (injection, modes A/B) and eCom keep-alive floor (brownout prevention, mode C).
15.1	Jun 2026	AART	STM32G051K6U6 adopted as current MCU. G051 has DAC for linear control, same QFN-32 package. Linker script renamed stm32g051k6.ld.

Rev	Date	Author	Changes
15.0	Jun 2026	AART	Q1/Q2 and GD1/GD2 renumbered to match topology diagram. D_PG pass gate added to BLACK path. PA6/PA7→PA4/PA5 DAC. All component tables updated.